

CSE 385: Analysis of Algorithms (Honors)

Lecture 4 (Divide-and-Conquer Algorithms: Merge Sort)

**Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026**

A Latin Phrase

“Divide et impera”

(meaning: “divide and rule” or “divide and conquer”)

— *Philip II, king of Macedon (382-336 BC),
describing his policy toward the Greek city-states
(some say the Roman emperor Julius Caesar,
100-44 BC, is the source of this phrase)*

The strategy is to break large power alliances into smaller ones that are easier to manage (or subdue).

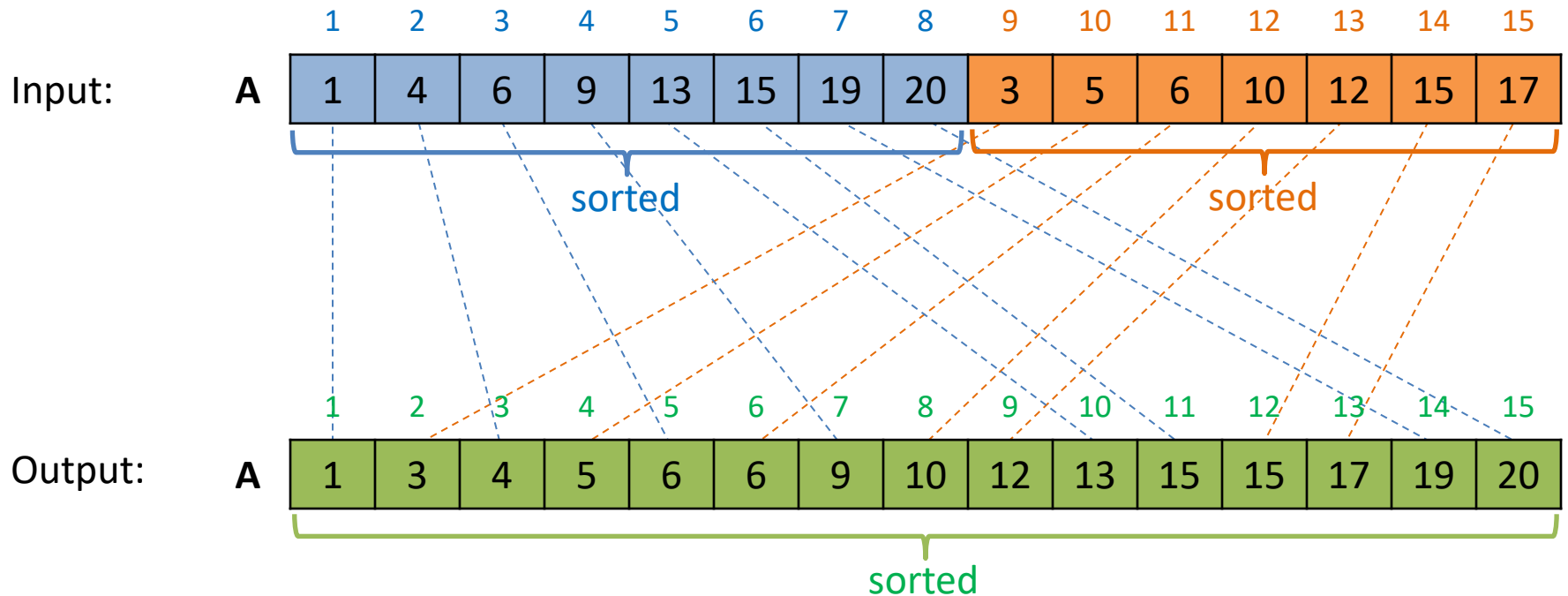
This is a combination of political, military and economic strategy of gaining and maintaining power.

Unsurprisingly, this is also a very powerful problem solving strategy in computer science.

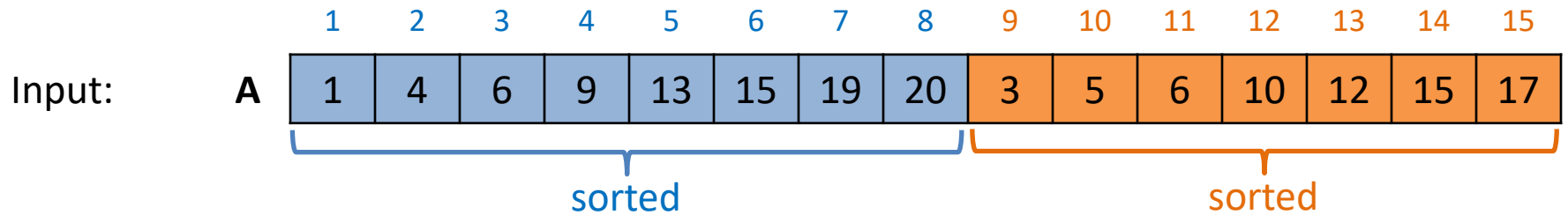
Divide-and-Conquer

1. **Divide:** divide the original problem into smaller subproblems that are easier to solve
2. **Conquer:** solve the smaller subproblems
(perhaps recursively)
3. **Merge:** combine the solutions to the smaller subproblems to obtain a solution for the original problem

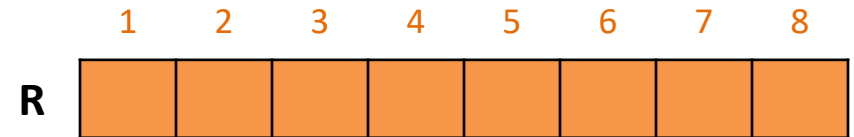
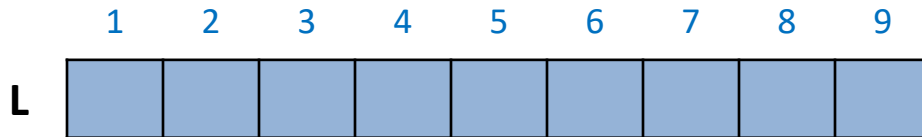
Merging Two Sorted Subarrays



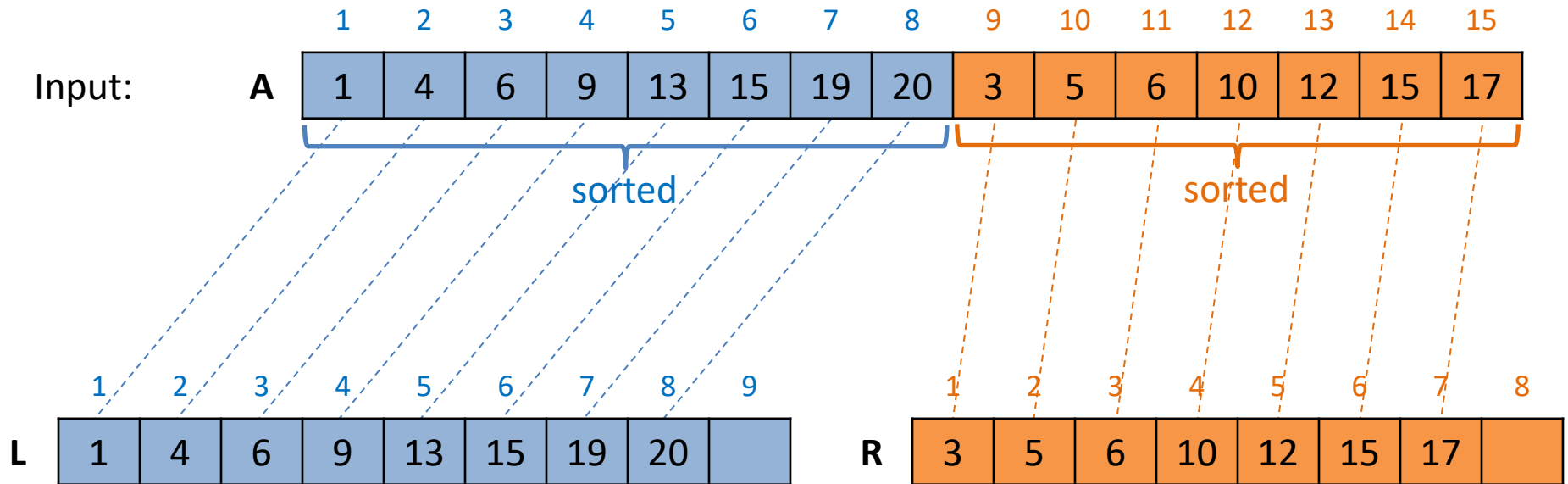
Merging Two Sorted Subarrays



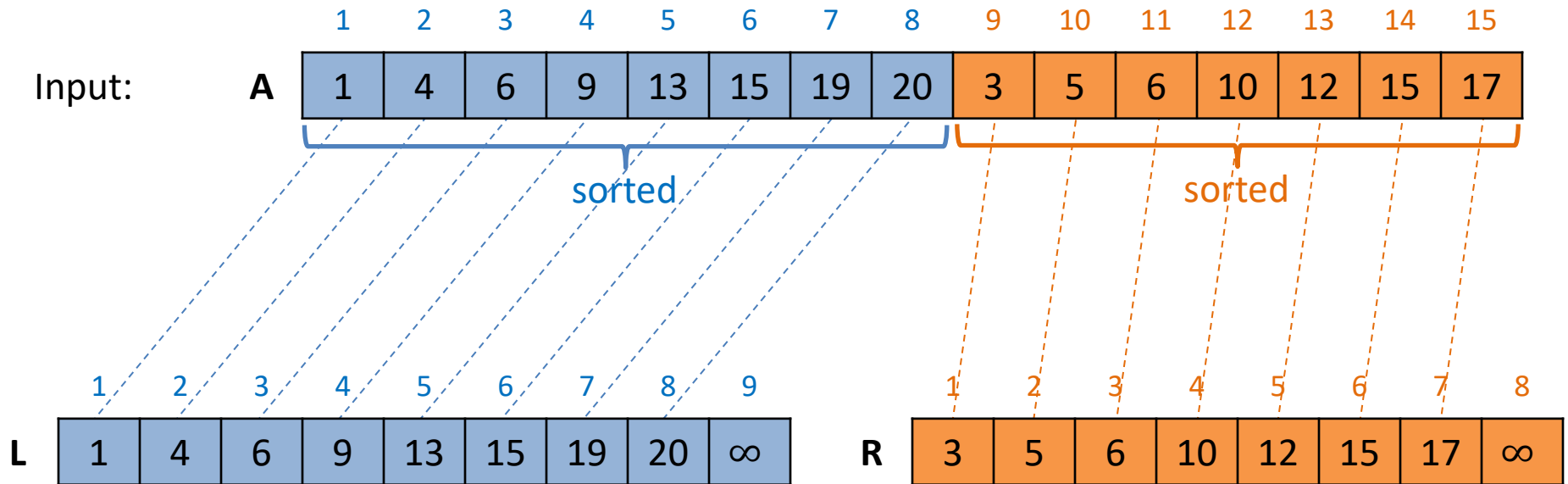
Temporary arrays:



Merging Two Sorted Subarrays



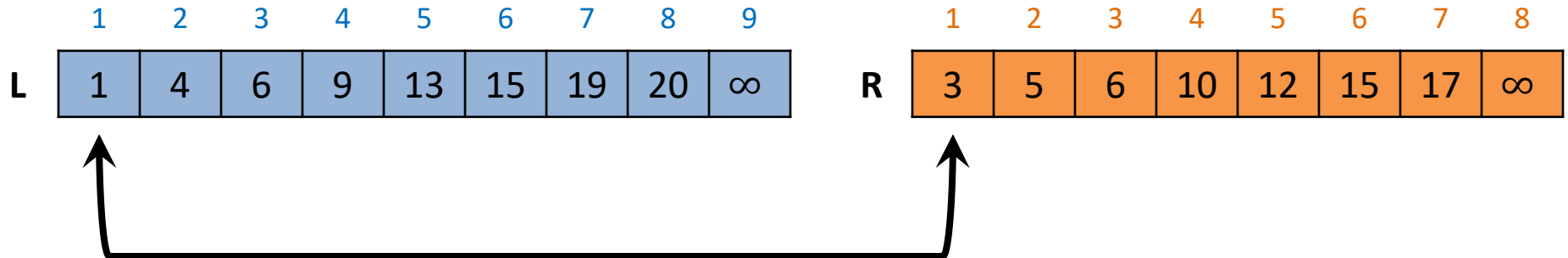
Merging Two Sorted Subarrays



Merging Two Sorted Subarrays

Output:

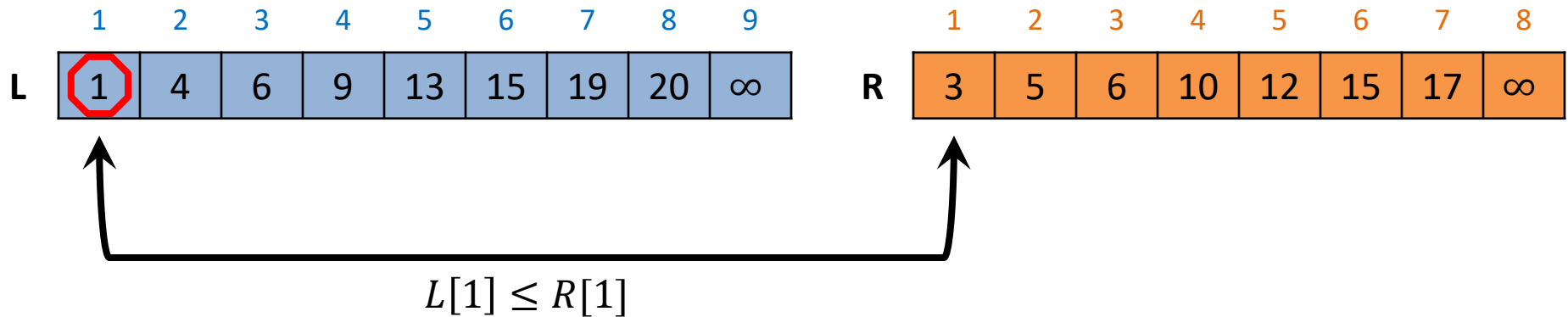
A	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20	3	5	6	10	12	15	17



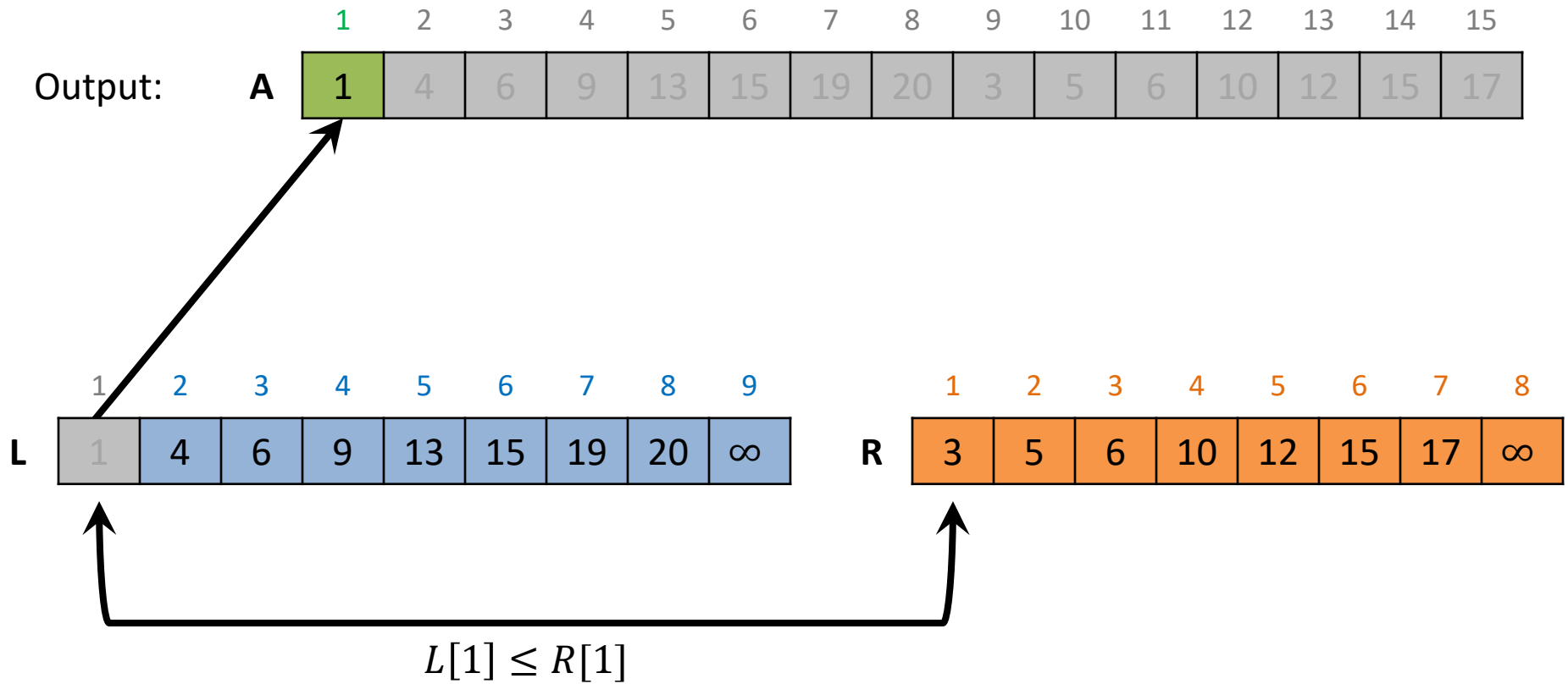
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	4	6	9	13	15	19	20	3	5	6	10	12	15	17



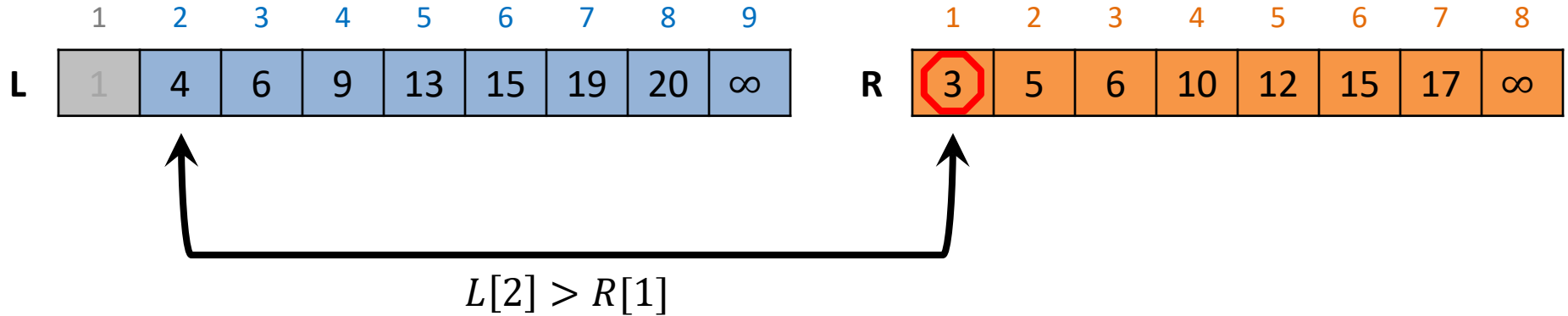
Merging Two Sorted Subarrays



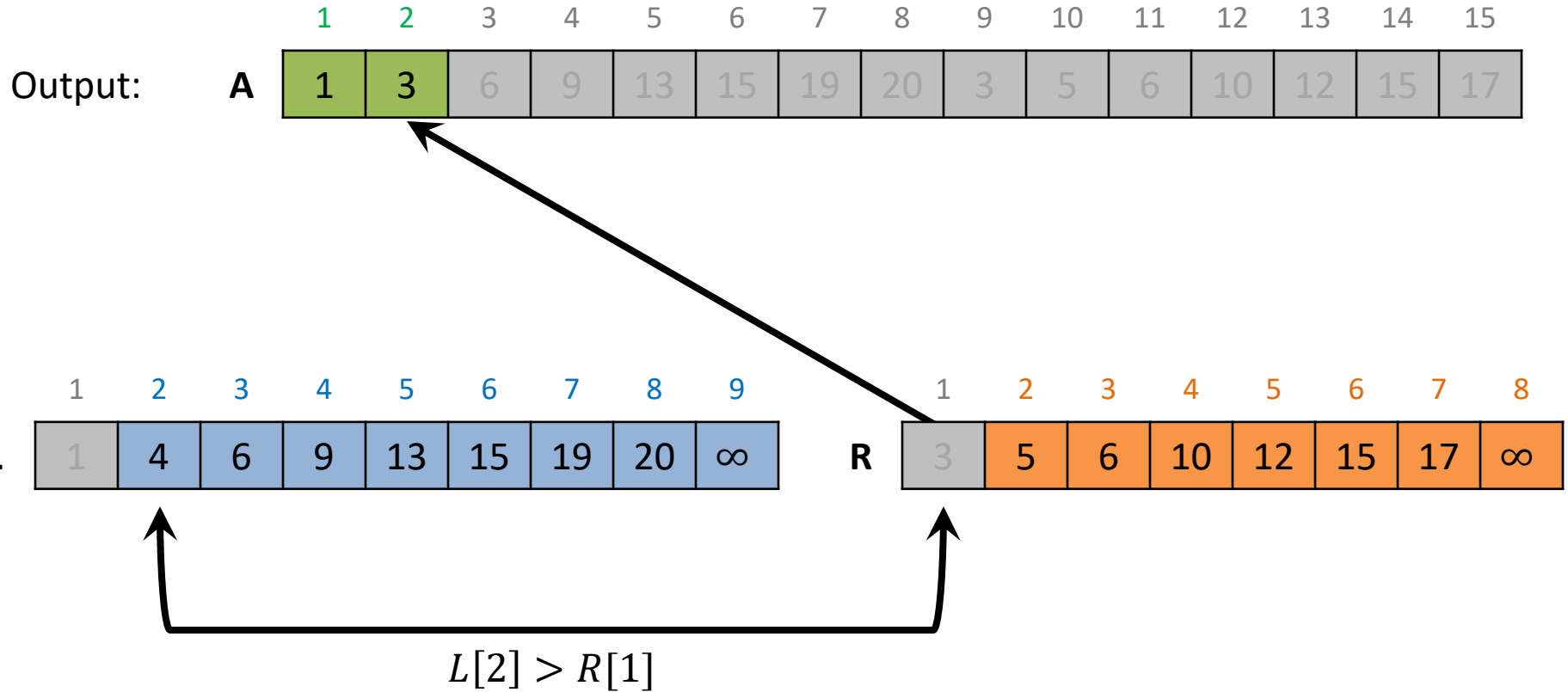
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	4	6	9	13	15	19	20	3	5	6	10	12	15	17



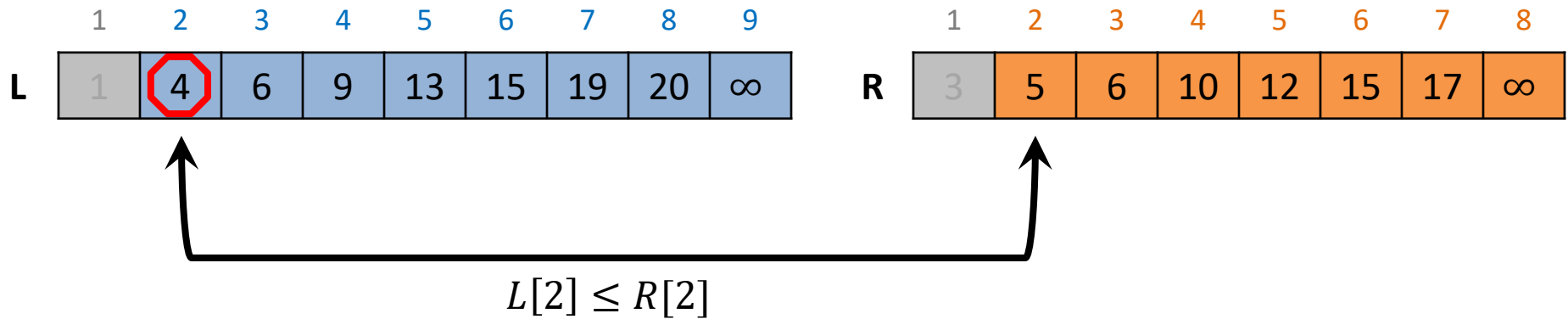
Merging Two Sorted Subarrays



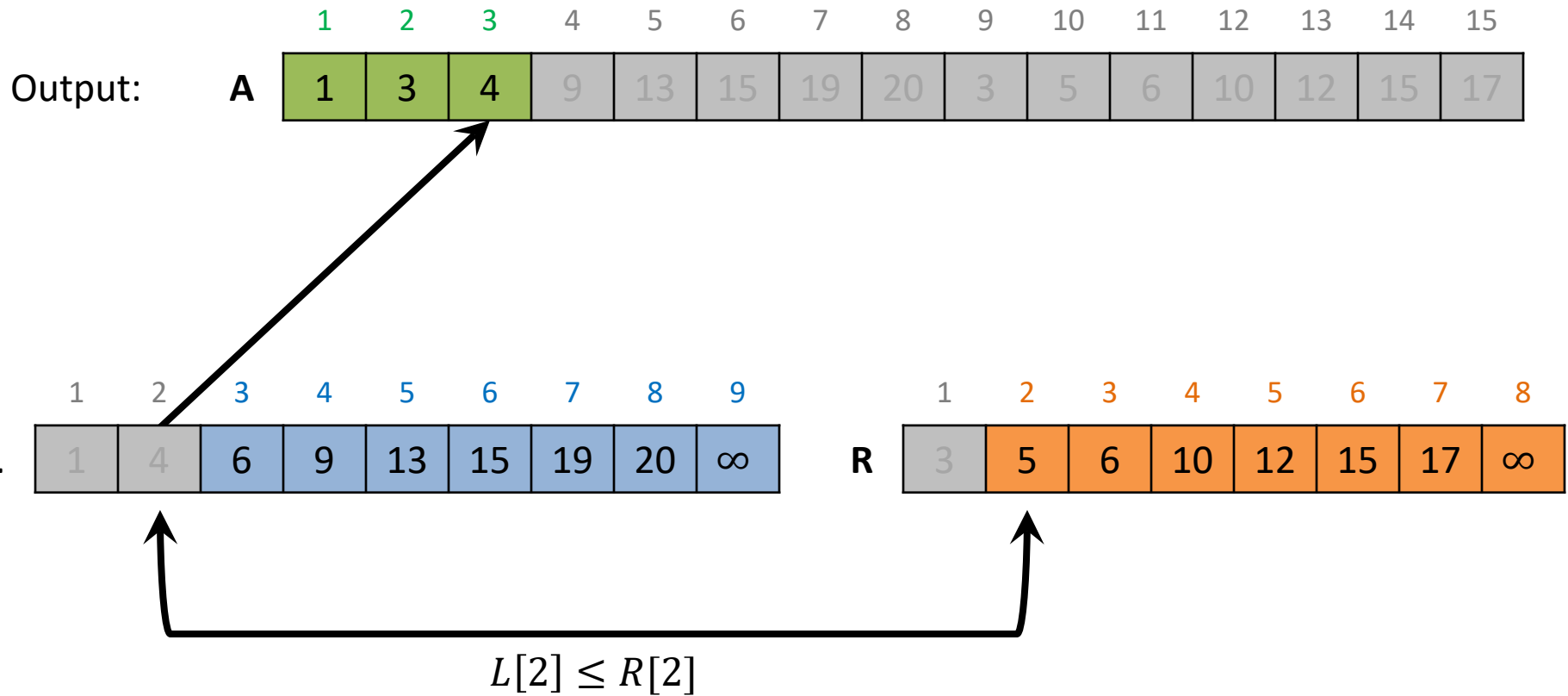
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	6	9	13	15	19	20	3	5	6	10	12	15	17



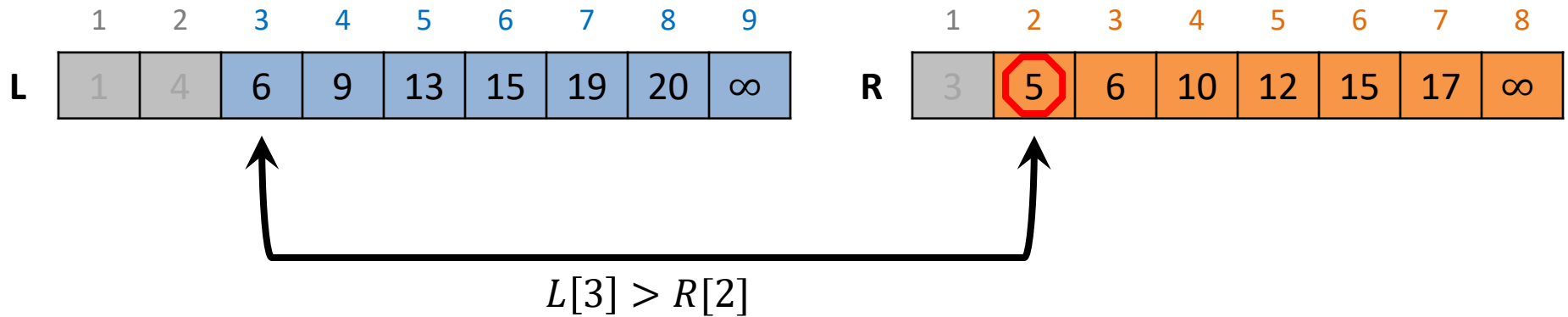
Merging Two Sorted Subarrays



Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	9	13	15	19	20	3	5	6	10	12	15	17



Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	13	15	19	20	3	5	6	10	12	15	17

	1	2	3	4	5	6	7	8	9
L	1	4	6	9	13	15	19	20	∞

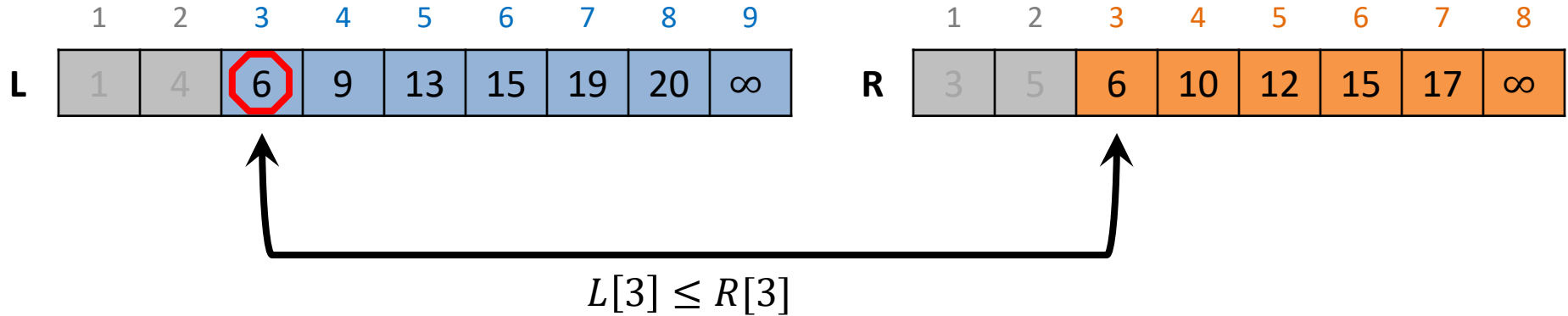
	1	2	3	4	5	6	7	8
R	3	5	6	10	12	15	17	∞

$L[3] > R[2]$

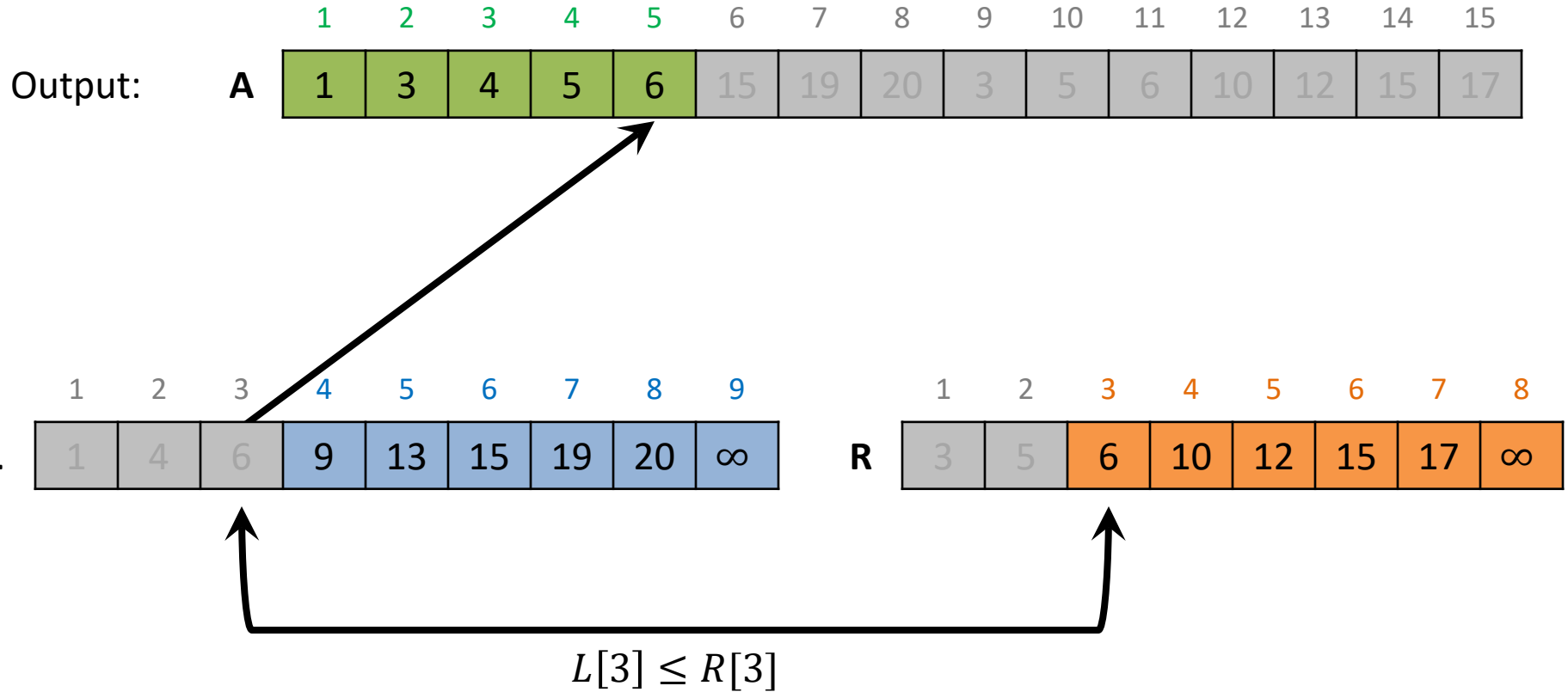
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	13	15	19	20	3	5	6	10	12	15	17



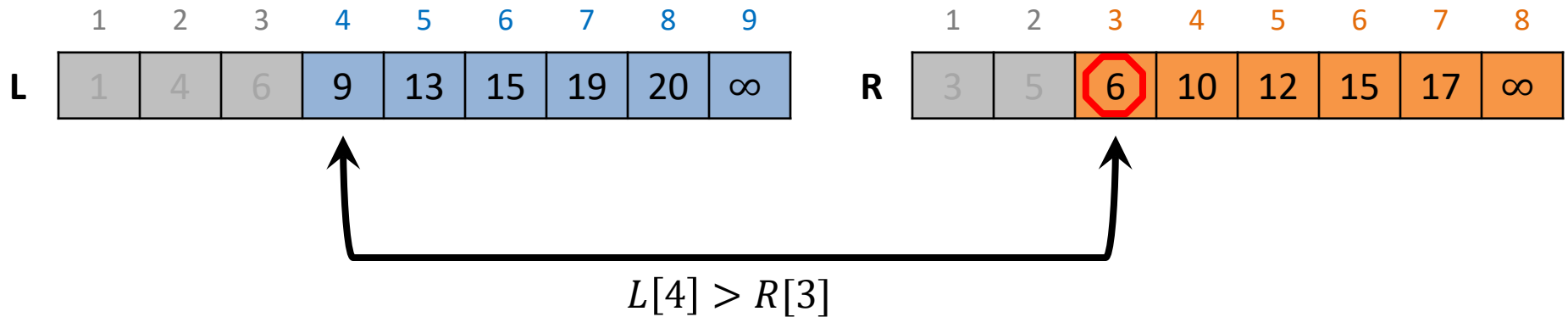
Merging Two Sorted Subarrays



Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	15	19	20	3	5	6	10	12	15	17



Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	19	20	3	5	6	10	12	15	17

	1	2	3	4	5	6	7	8	9
L	1	4	6	9	13	15	19	20	∞

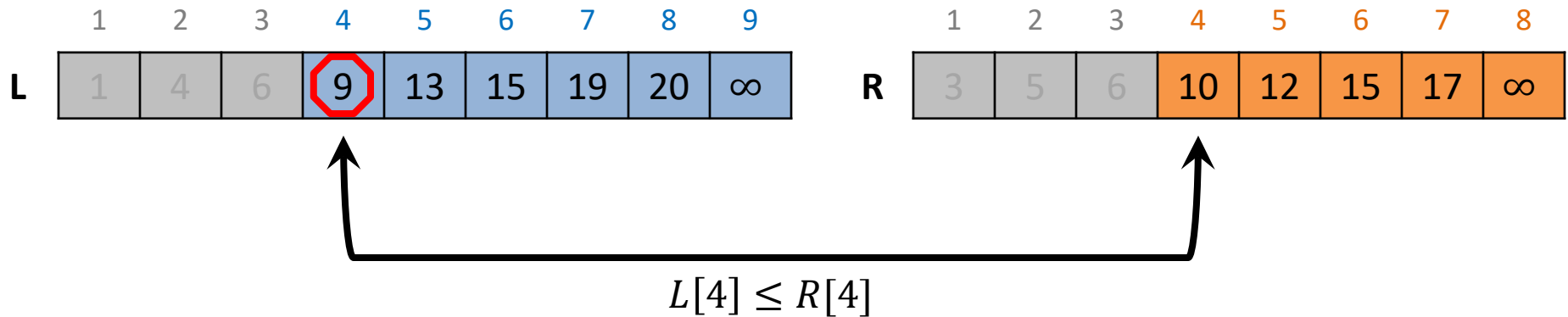
	1	2	3	4	5	6	7	8
R	3	5	6	10	12	15	17	∞

$L[4] > R[3]$

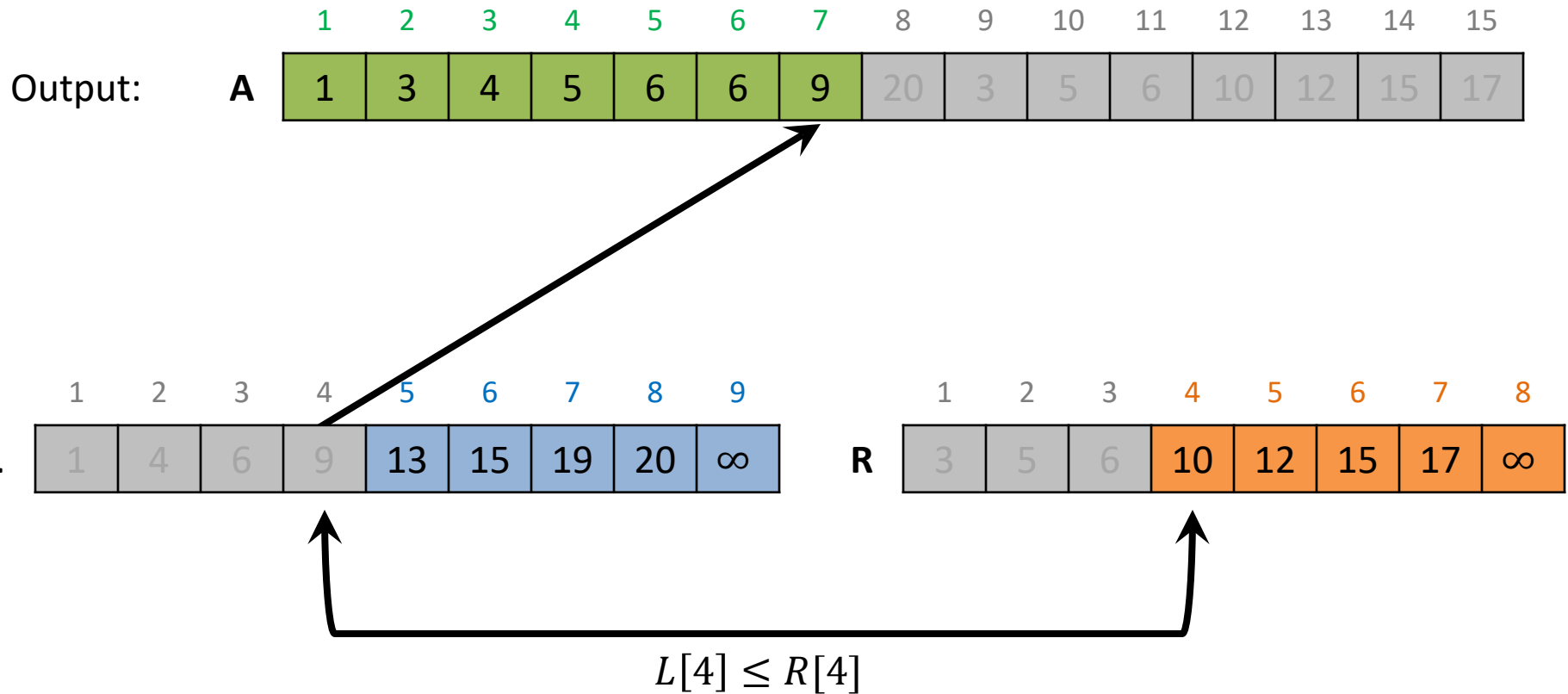
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	19	20	3	5	6	10	12	15	17



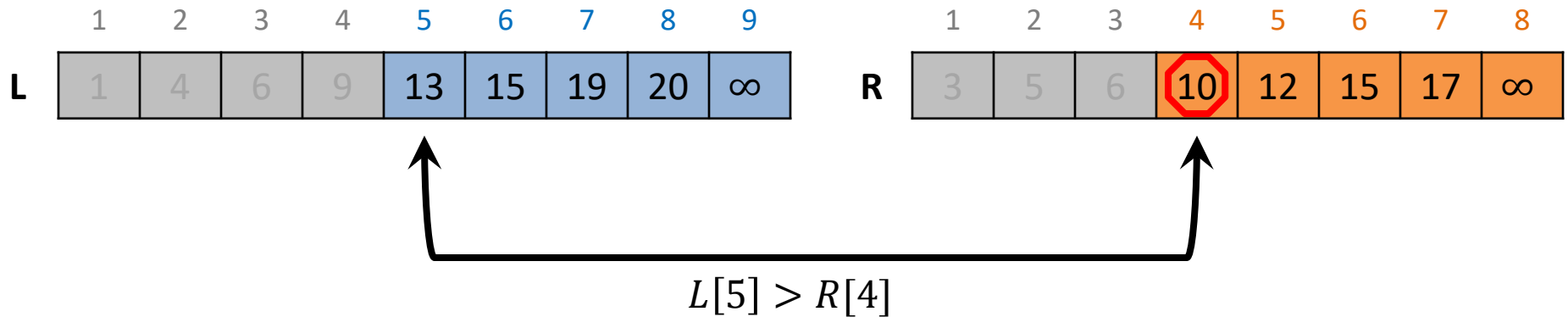
Merging Two Sorted Subarrays



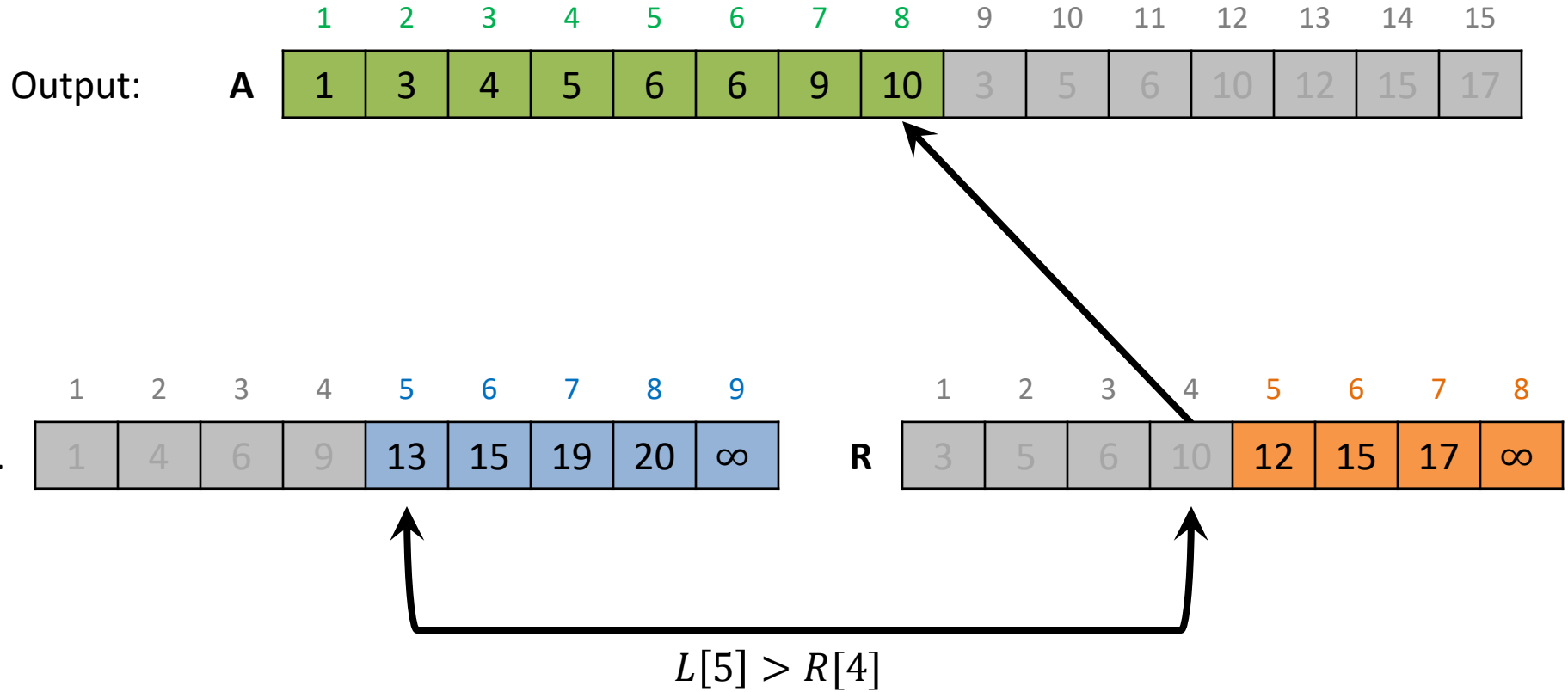
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	20	3	5	6	10	12	15	17



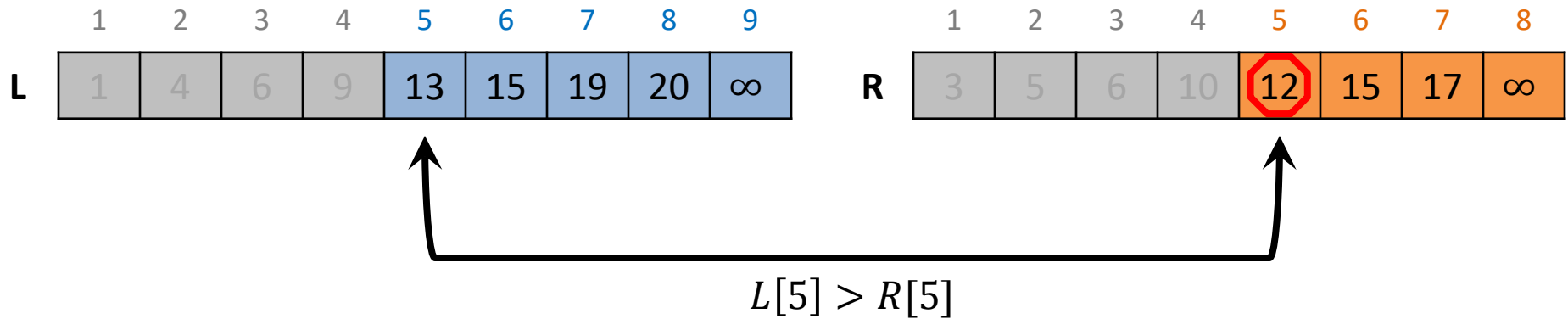
Merging Two Sorted Subarrays



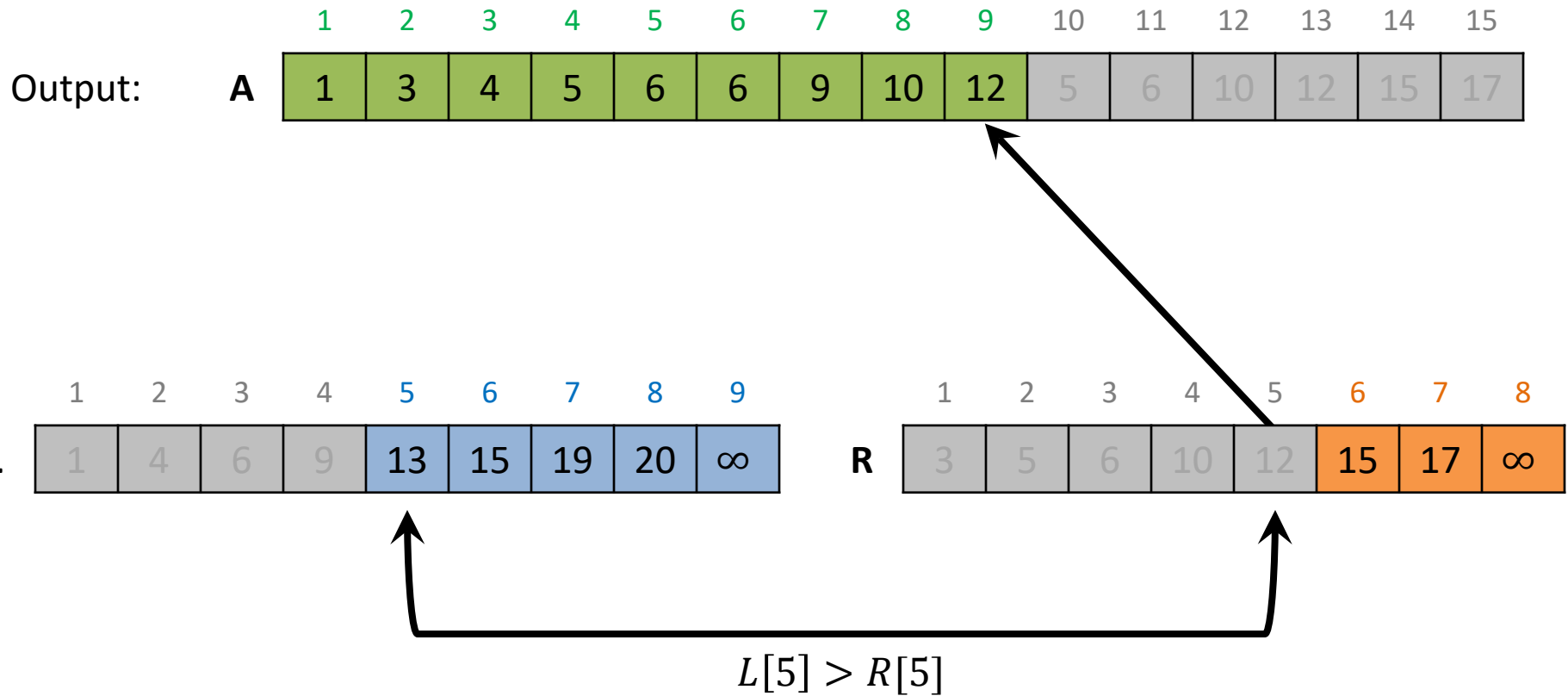
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	3	5	6	10	12	15	17



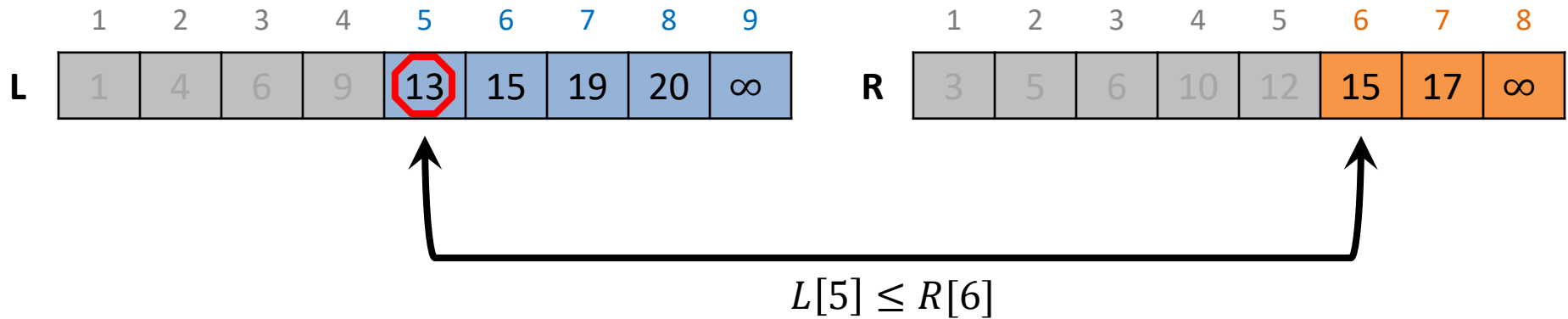
Merging Two Sorted Subarrays



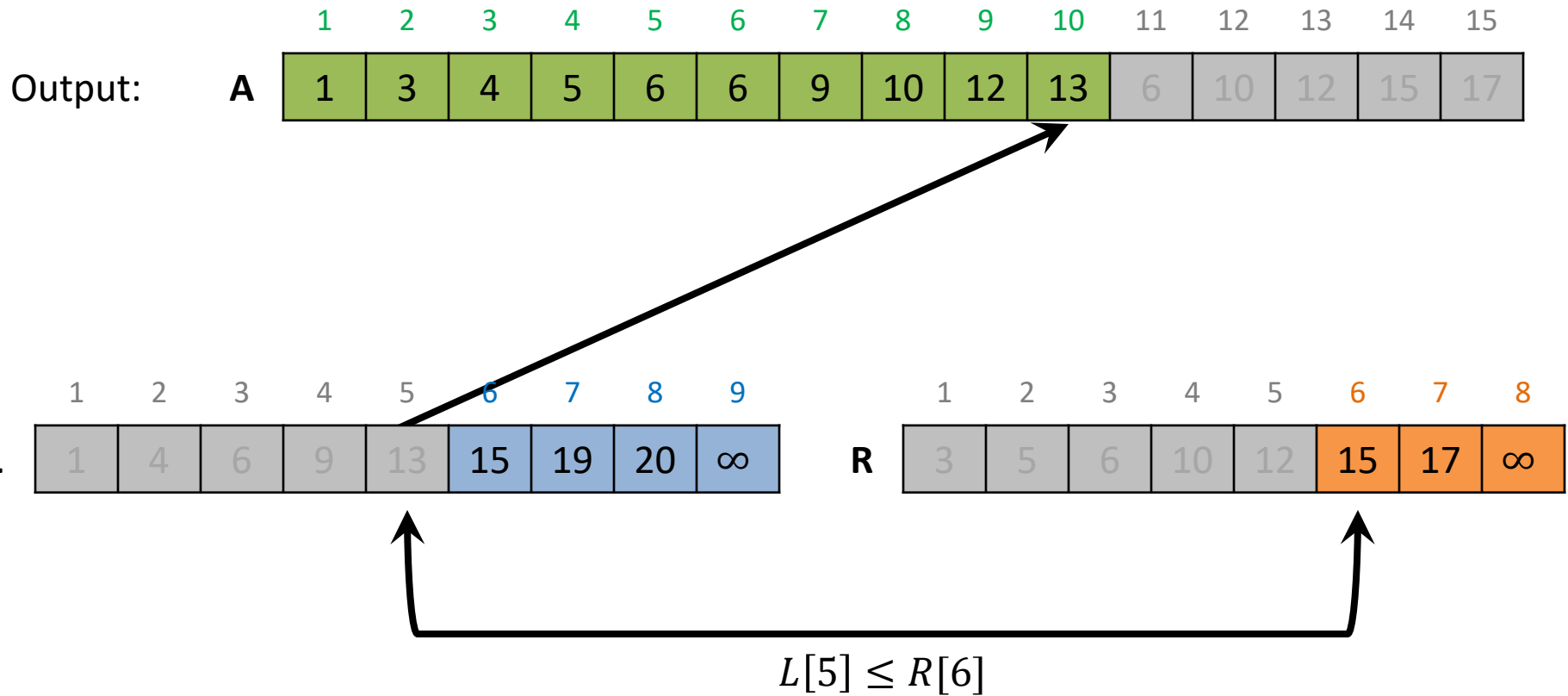
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	5	6	10	12	15	17



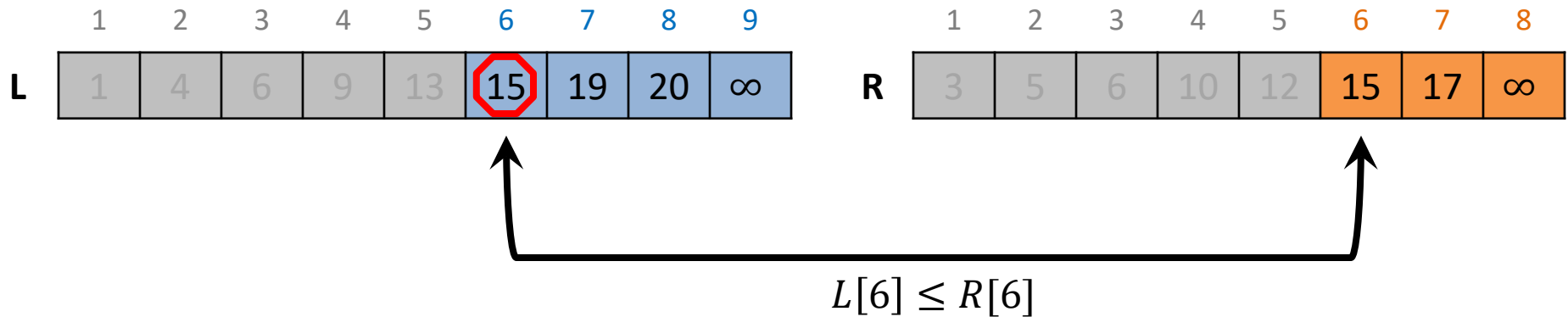
Merging Two Sorted Subarrays



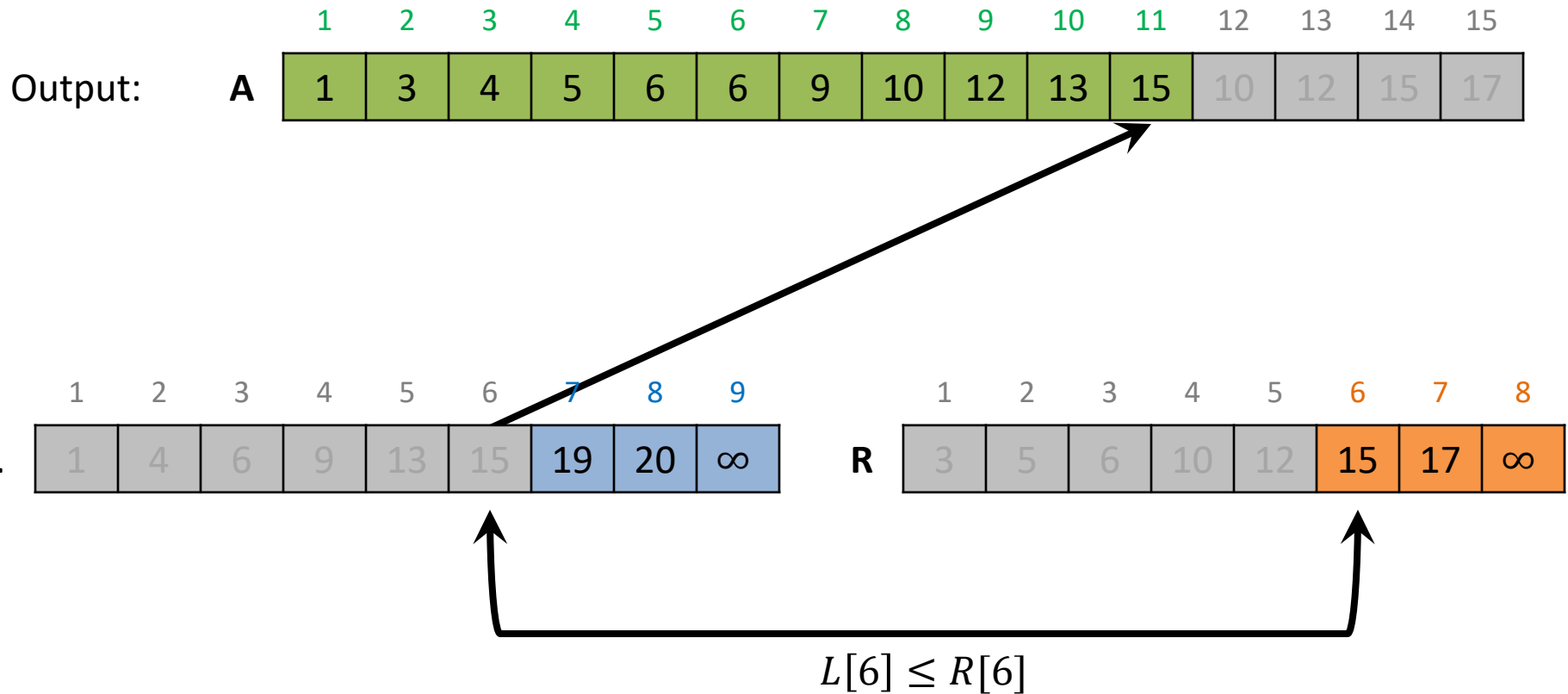
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	6	10	12	15	17



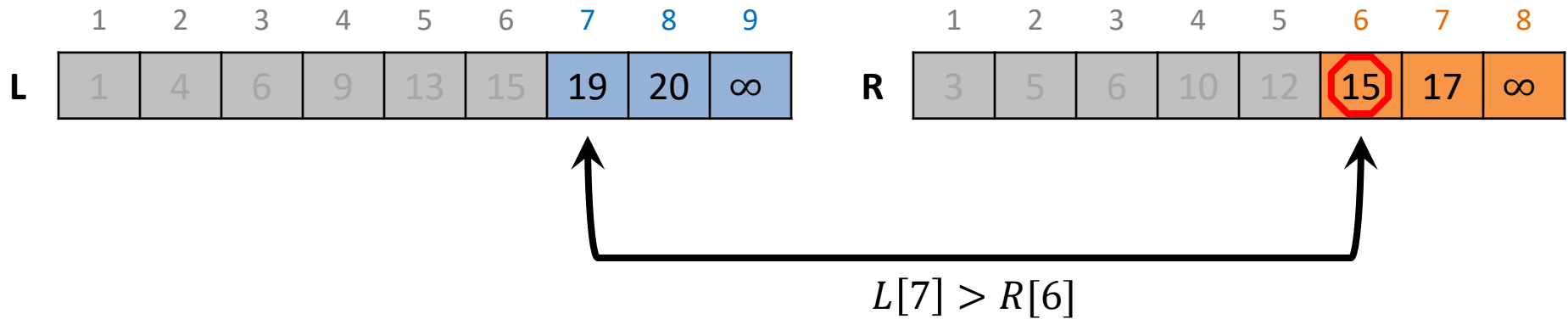
Merging Two Sorted Subarrays



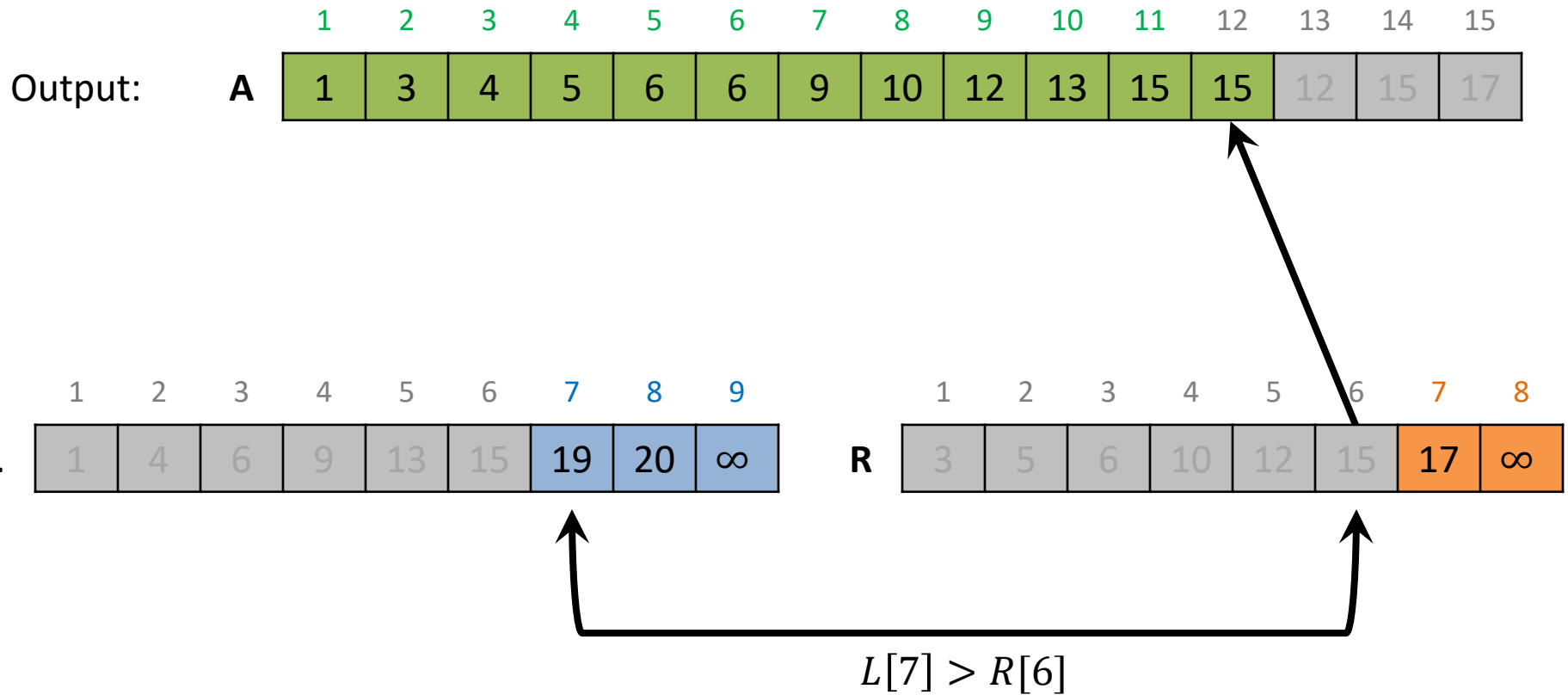
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	10	12	15	17



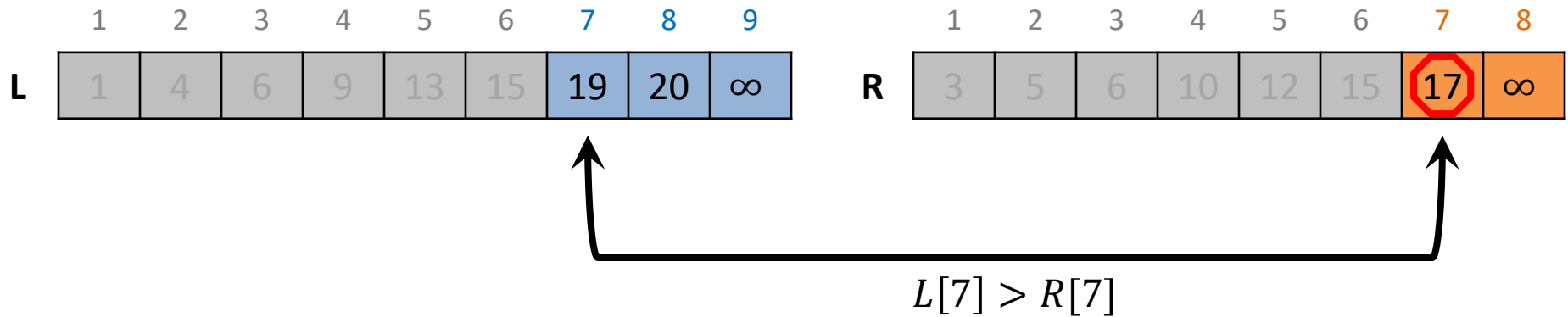
Merging Two Sorted Subarrays



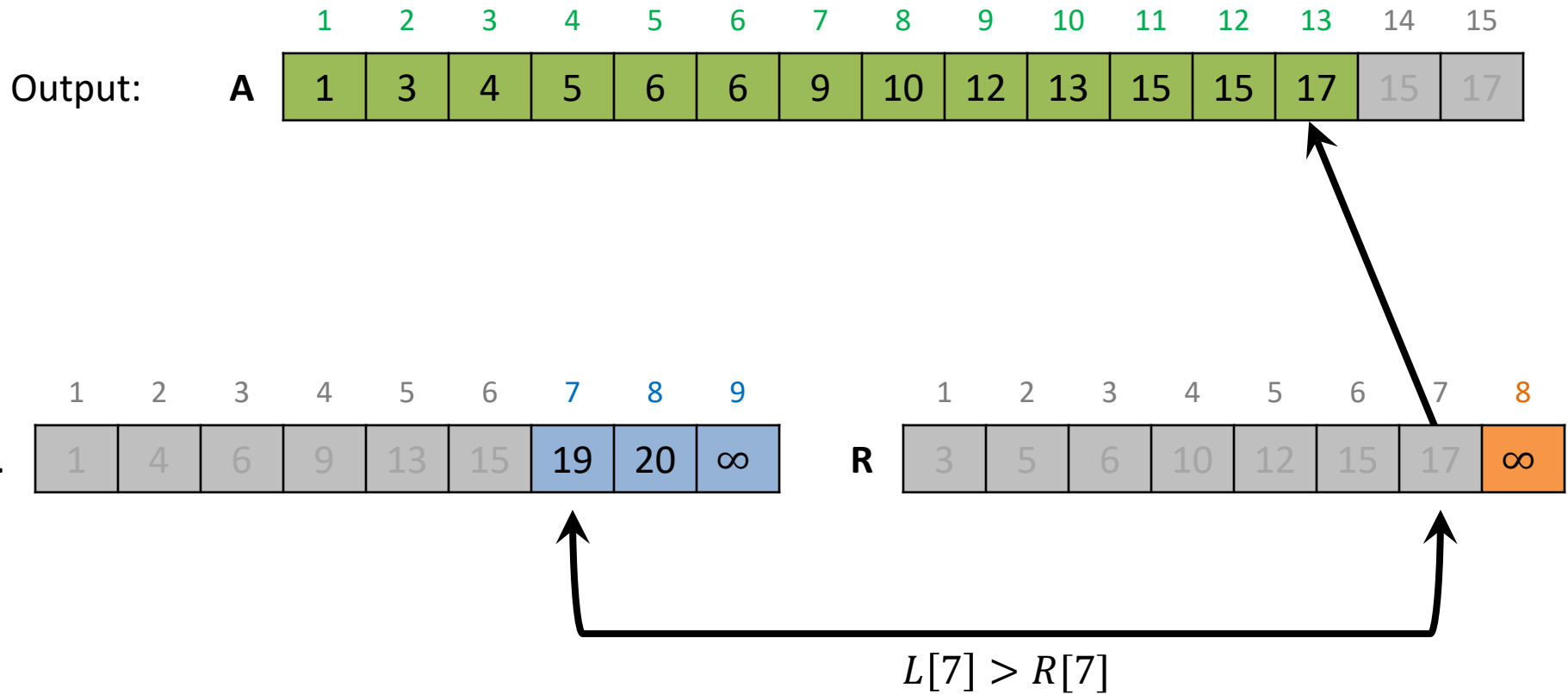
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	15	12	15	17



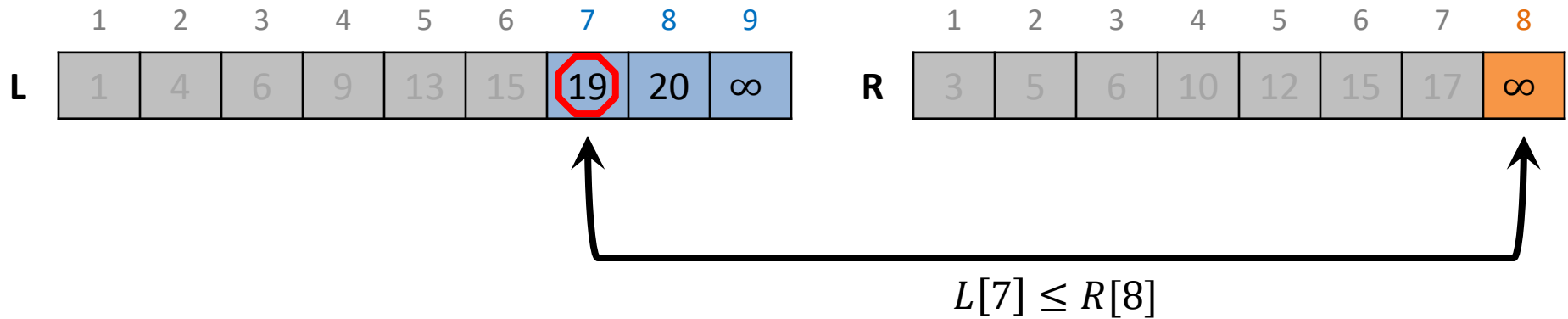
Merging Two Sorted Subarrays



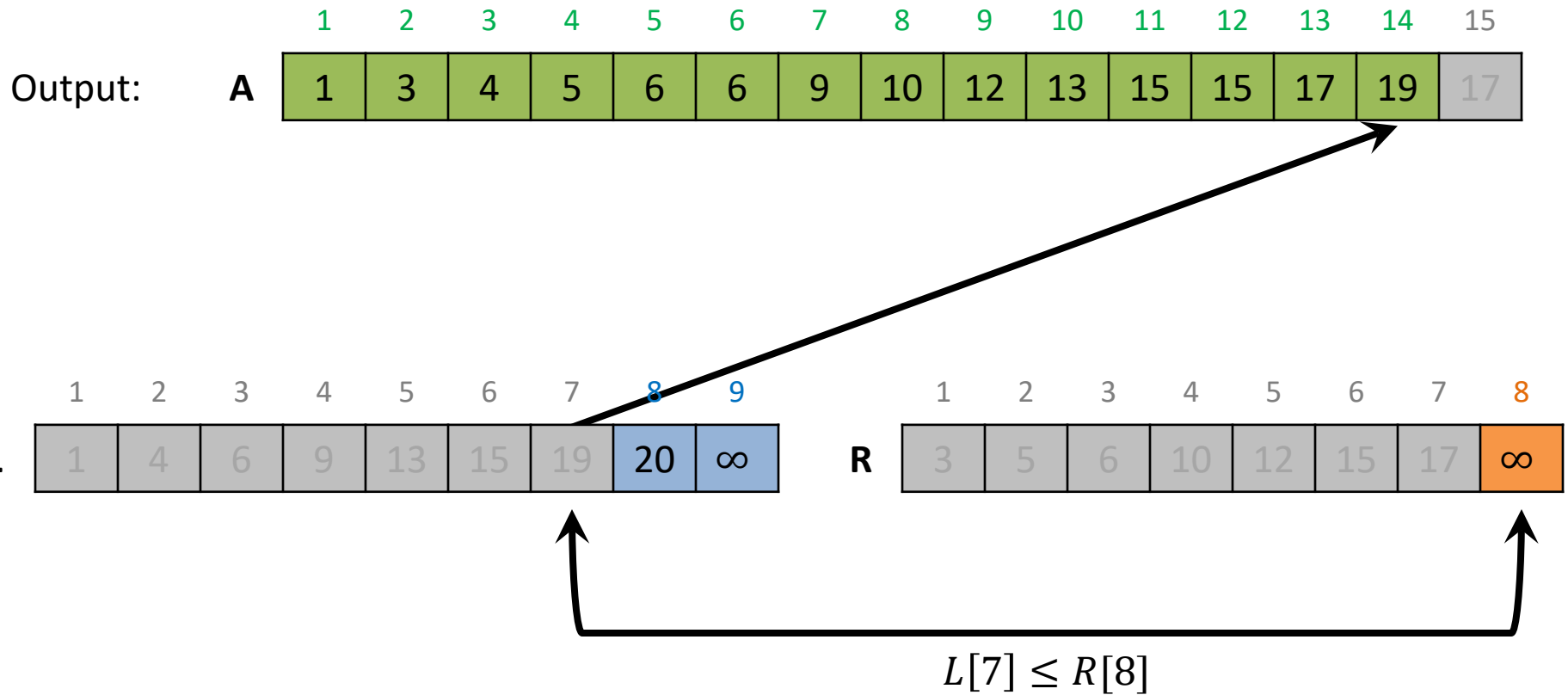
Merging Two Sorted Subarrays

Output:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	15	17	15	17



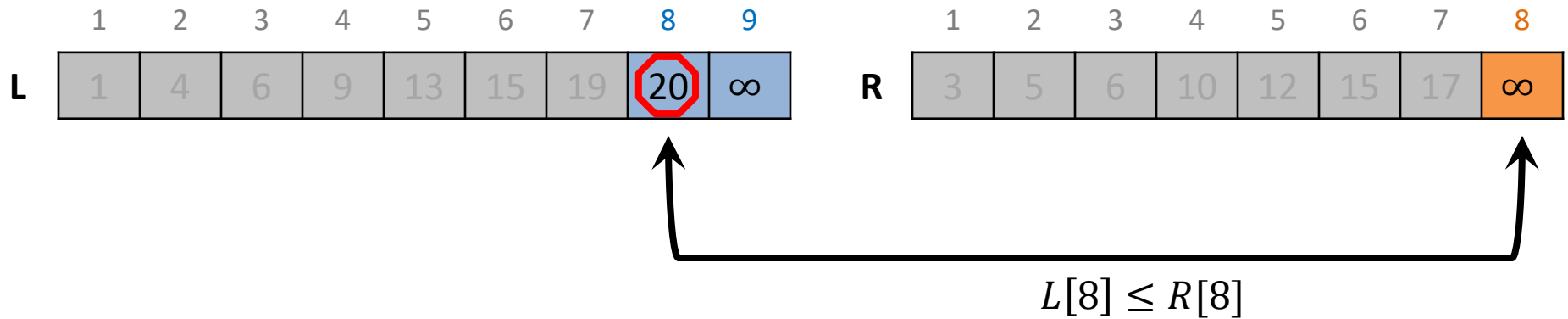
Merging Two Sorted Subarrays



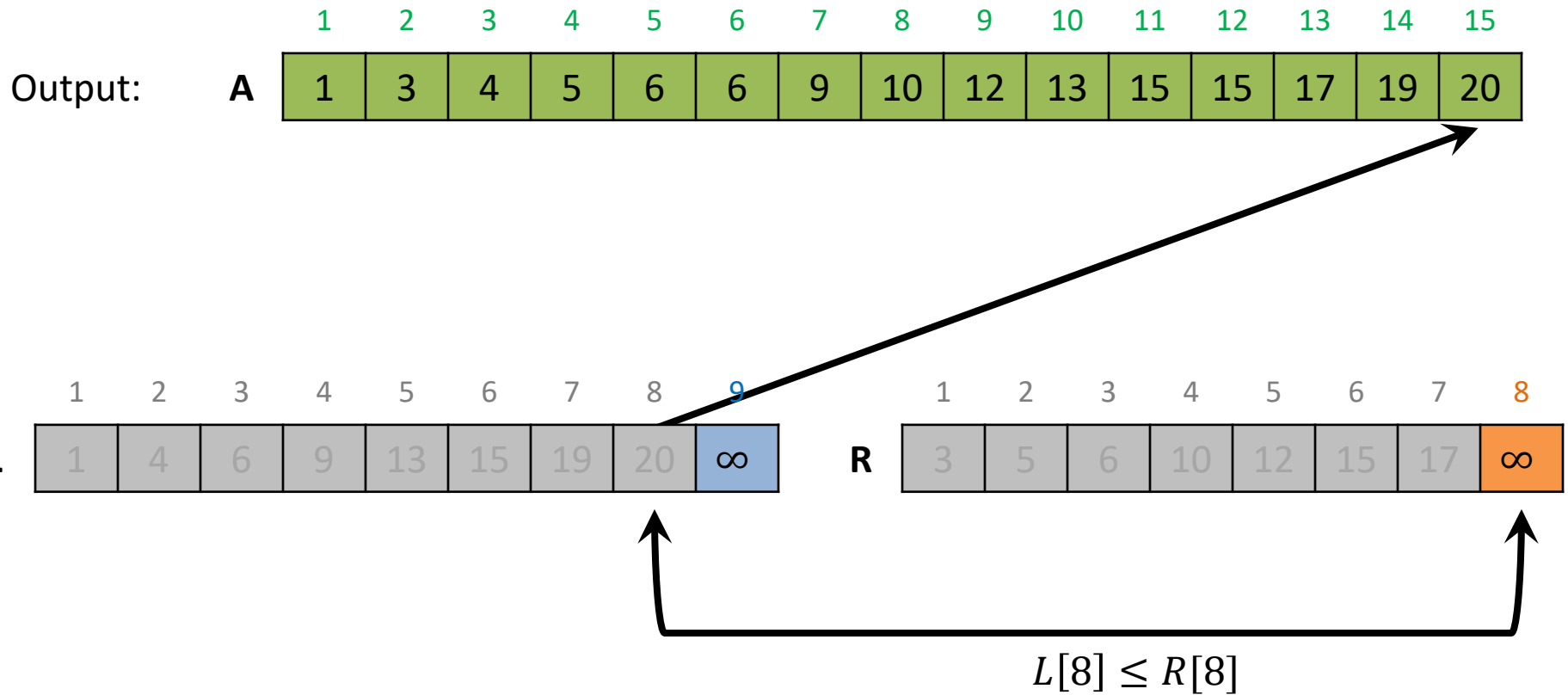
Merging Two Sorted Subarrays

Output:

A	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	1	3	4	5	6	6	9	10	12	13	15	15	17	19	17



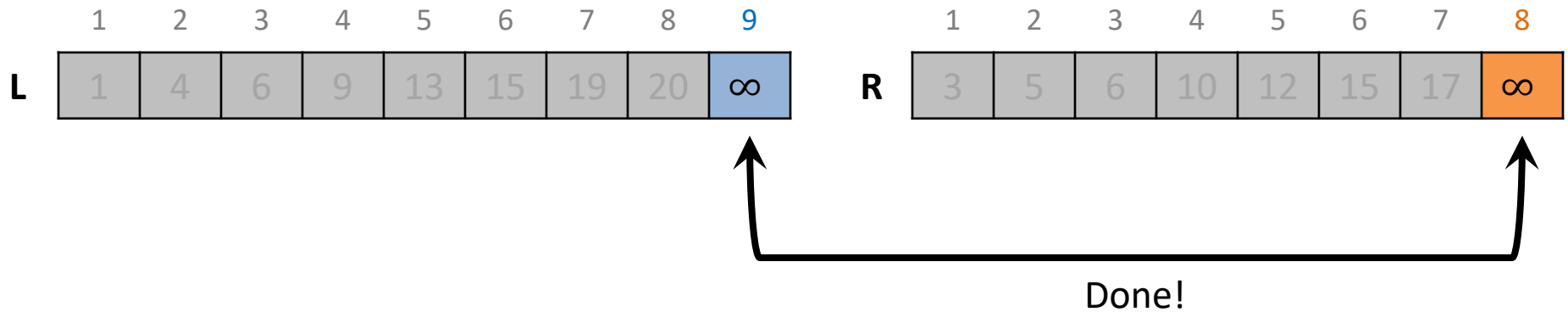
Merging Two Sorted Subarrays



Merging Two Sorted Subarrays

Output:

A	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Let $n = r - p + 1$.
Then $n = n_1 + n_2$.

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Let $n = r - p + 1$.

Then $n = n_1 + n_2$.

The loop in lines 4–5 takes $\Theta(n_1)$ time.

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Let $n = r - p + 1$.

Then $n = n_1 + n_2$.

The loop in lines 4–5 takes $\Theta(n_1)$ time.

The loop in lines 6–7 takes $\Theta(n_2)$ time.

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1 : n_1 + 1]$ and $R[1 : n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Let $n = r - p + 1$.

Then $n = n_1 + n_2$.

The loop in lines 4–5 takes $\Theta(n_1)$ time.

The loop in lines 6–7 takes $\Theta(n_2)$ time.

The loop in lines 12–17 takes $\Theta(n)$ time.

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1:n_1 + 1]$ and $R[1:n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Running Time

Let $n = r - p + 1$.

Then $n = n_1 + n_2$.

The loop in lines 4–5 takes $\Theta(n_1)$ time.

The loop in lines 6–7 takes $\Theta(n_2)$ time.

The loop in lines 12–17 takes $\Theta(n)$ time.

Lines 1–3 and 8–11 take $\Theta(1)$ time.

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

```
1.   $n_1 = q - p + 1$ 
2.   $n_2 = r - q$ 
3.  Let  $L[1:n_1 + 1]$  and  $R[1:n_2 + 1]$  be new arrays
4.  for  $i = 1$  to  $n_1$ 
5.       $L[i] = A[p + i - 1]$ 
6.  for  $j = 1$  to  $n_2$ 
7.       $R[j] = A[q + j]$ 
8.   $L[n_1 + 1] = \infty$ 
9.   $R[n_2 + 1] = \infty$ 
10.  $i = 1$ 
11.  $j = 1$ 
12. for  $k = p$  to  $r$ 
13.     if  $L[i] \leq R[j]$ 
14.          $A[k] = L[i]$ 
15.          $i = i + 1$ 
16.     else  $A[k] = R[j]$ 
17.          $j = j + 1$ 
```

Running Time

Let $n = r - p + 1$.

Then $n = n_1 + n_2$.

The loop in lines 4–5 takes $\Theta(n_1)$ time.

The loop in lines 6–7 takes $\Theta(n_2)$ time.

The loop in lines 12–17 takes $\Theta(n)$ time.

Lines 1–3 and 8–11 take $\Theta(1)$ time.

Overall running time

$$= \Theta(n_1) + \Theta(n_2) + \Theta(n) + \Theta(1)$$

$$= \Theta(n)$$

Divide-and-Conquer

1. **Divide:** divide the original problem into smaller subproblems that are easier to solve
2. **Conquer:** solve the smaller subproblems
(perhaps recursively)
3. **Merge:** combine the solutions to the smaller subproblems to obtain a solution for the original problem

Intuition Behind Merge Sort

1. **Base case:** We know how to correctly sort an array containing only a single element.

Indeed, an array of one number is already trivially sorted!

2. **Reduction to base case (recursive divide and conquer):**

At each level of recursion we split the current subarray at the midpoint (approx) to obtain two subsubarrays of equal or almost equal lengths, and sort them recursively.

We are guaranteed to reach subproblems of size 1 (i.e., the base case size) eventually which are trivially sorted.

3. **Merge:** We know how to merge two (recursively) sorted subarrays to obtain a longer sorted subarray.

Merge Sort

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

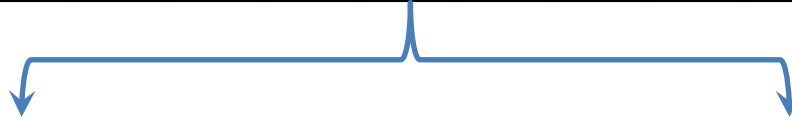
A

9	1	13	20	4	15	19	6	12	6	3	17	15	5	10
---	---	----	----	---	----	----	---	----	---	---	----	----	---	----

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



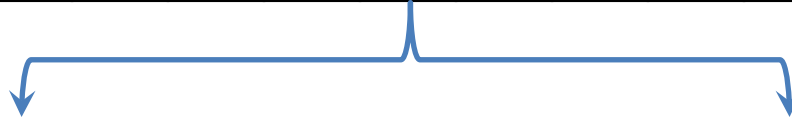
Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Merge Sort

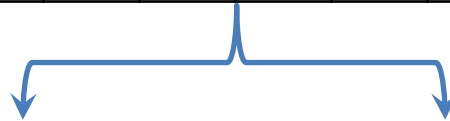
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10



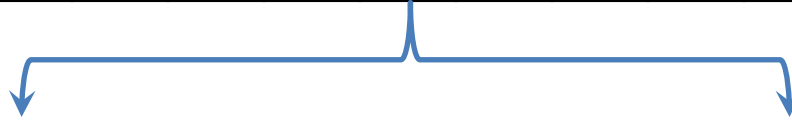
Split:

	1	2	3	4		5	6	7	8
	9	1	13	20		4	15	19	6

Merge Sort

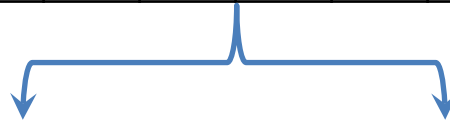
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



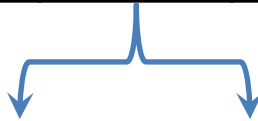
Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10



Split:

	1	2	3	4		5	6	7	8
	9	1	13	20		4	15	19	6



Split:

	1	2		3	4
	9	1		13	20

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8
	9	1	13	20	4	15	19	6

Split:

	1	2	3	4
	9	1	13	20

Split:

	1	2
	9	1

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
9	1	13	20	4	15	19	6

Split:

1	2	3	4
9	1	13	20

Split:

1	2
9	1

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	9	1	13	20		4	15	19	6

Split:

	1	2		3	4
	9	1		13	20

Merge:

	1	2
	9	1

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	9	1	13	20		4	15	19	6

Split:

	1	2		3	4
	1	9		13	20

Merge:

	1	2
	9	1

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
9	1	13	20	4	15	19	6

Split:

1	2	3	4
1	9	13	20

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
9	1	13	20	4	15	19	6

Split:

1	2	3	4
1	9	13	20

Split:

3	4
13	20

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	9	1	13	20		4	15	19	6

Split:

	1	2		3	4
	1	9		13	20

Split:

	3	4
	13	20

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8
	9	1	13	20	4	15	19	6

Split:

	1	2	3	4
	1	9	13	20

Merge:

	3	4
	13	20

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8
	9	1	13	20	4	15	19	6

Split:

	1	2	3	4
	1	9	13	20

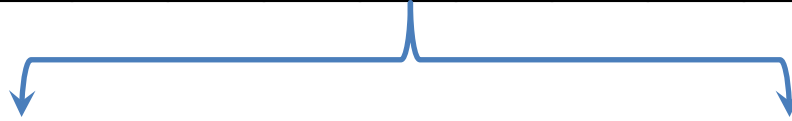
Merge:

	3	4
	13	20

Merge Sort

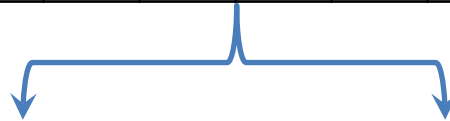
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



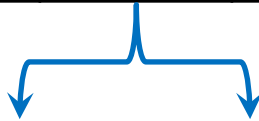
Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

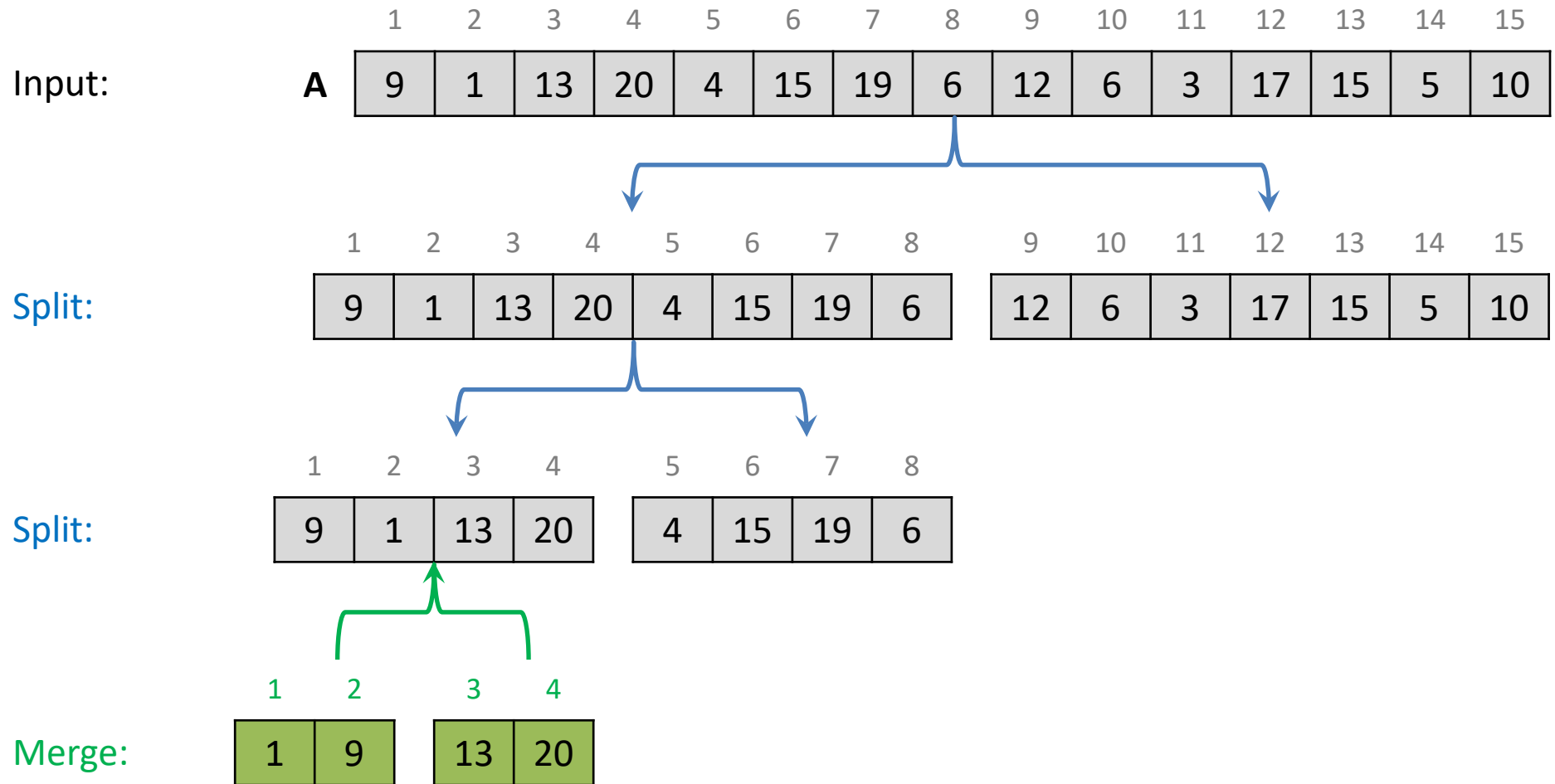
	1	2	3	4	5	6	7	8
	9	1	13	20	4	15	19	6



Split:

	1	2	3	4
	1	9	13	20

Merge Sort



Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8
	1	9	13	20	4	15	19	6

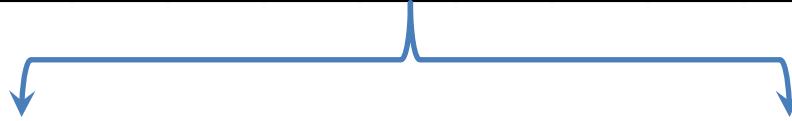
Merge:

	1	2	3	4
	1	9	13	20

Merge Sort

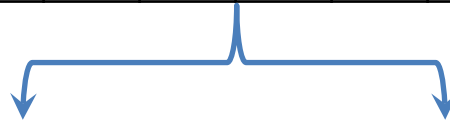
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	19	6

5	6	7	8
4	15	19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	15	19	6

Split:

	5	6		7	8
	4	15		19	6

Split:

	5		6
	4		15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	15	19	6

Split:

	5	6		7	8
	4	15		19	6

Split:

	5		6
	4		15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	19	6

Merge:

5	6
4	15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	15	19	6

Split:

			5	6		7	8
			4	15		19	6

Merge:

			5			6
			4			15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10

Split:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	15	19	6

Split:

	5	6		7	8
	4	15		19	6

Split:

	7	8
	19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	19	6

Split:

7	8
19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	19	6

Merge:

7	8
19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	6	19

Merge:

7	8
19	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Split:

5	6	7	8
4	15	6	19

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	15	19	6

Merge:

5	6	7	8
4	15	6	19

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8
1	9	13	20	4	6	15	19

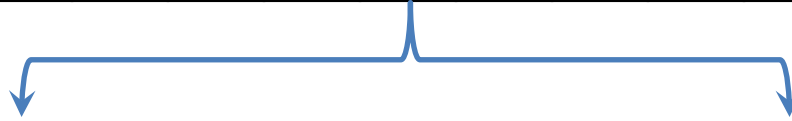
Merge:

5	6	7	8
4	15	6	19

Merge Sort

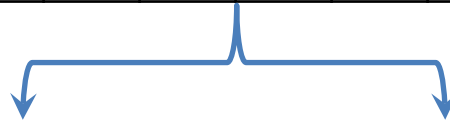
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10



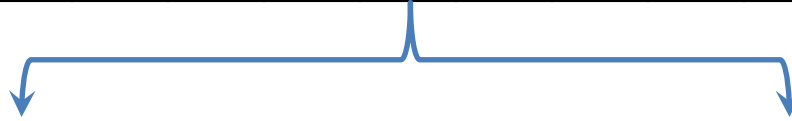
Split:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	6	15	19

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6		12	6	3	17	15	5	10



Merge:

	1	2	3	4		5	6	7	8
	1	9	13	20		4	6	15	19

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

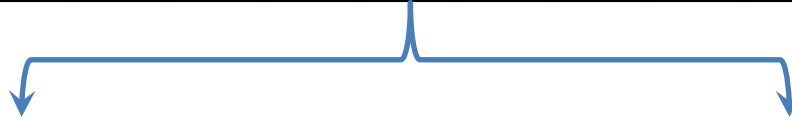
Merge:

1	2	3	4	5	6	7	8
1	9	13	20	4	6	15	19

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
12	6	3	17	15	5	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
12	6	3	17	15	5	10

Split:

9	10	11	12
12	6	3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	12	6		3	17

Split:

	9	10
	12	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	12	6		3	17

Split:

	9	10
	12	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	12	6		3	17

Merge:

	9	10
	12	6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	6	12		3	17

Merge:

	9		10
	12		6

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
12	6	3	17	15	5	10

Split:

9	10	11	12
6	12	3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	6	12		3	17

Split:

	11	12
	3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	6	12		3	17

Split:

	11	12
	3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
12	6	3	17	15	5	10

Split:

9	10	11	12
6	12	3	17

Merge:

11	12
3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Split:

	9	10		11	12
	6	12		3	17

Merge:

	11	12
	3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	12	6	3	17		15	5	10

Merge:

	9	10		11	12
	6	12		3	17

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		15	5	10

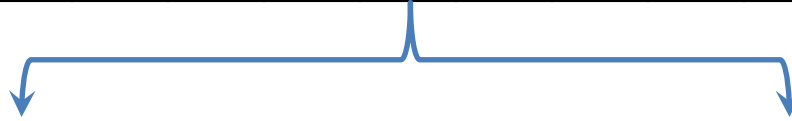
Merge:

	9	10		11	12
	6	12		3	17

Merge Sort

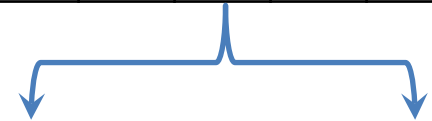
Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20	12	6	3	17	15	5	10



Split:

	9	10	11	12	13	14	15
	3	6	12	17	15	5	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		15	5	10

Split:

	13	14		15
	15	5		10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
3	6	12	17	15	5	10

Split:

13	14	15
15	5	10

Split:

13	14
15	5

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
3	6	12	17	15	5	10

Split:

13	14	15
15	5	10

Split:

13	14
15	5

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
3	6	12	17	15	5	10

Split:

13	14	15
15	5	10

Merge:

13	14
15	5

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
3	6	12	17	15	5	10

Split:

13	14	15
5	15	10

Merge:

13	14
15	5

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		15	5	10

Split:

	13	14	15
	5	15	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		15	5	10

Split:

	13	14	15
	5	15	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Split:

9	10	11	12	13	14	15
3	6	12	17	15	5	10

Merge:

13	14	15
5	15	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		5	10	15

Merge:

	13	14	15
	5	15	10

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

	1	2	3	4	5	6	7	8		9	10	11	12	13	14	15
	1	4	6	9	13	15	19	20		12	6	3	17	15	5	10

Split:

	9	10	11	12		13	14	15
	3	6	12	17		5	10	15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	12	6	3	17	15	5	10

Merge:

9	10	11	12	13	14	15
3	6	12	17	5	10	15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	3	5	6	10	12	15	17

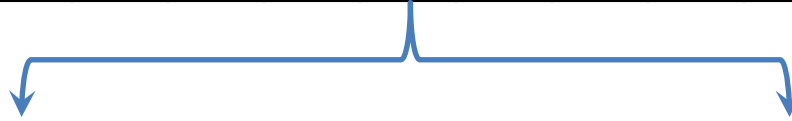
Merge:

9	10	11	12	13	14	15
3	6	12	17	5	10	15

Merge Sort

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



Split:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	3	5	6	10	12	15	17

Merge Sort

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10

Merge:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	3	5	6	10	12	15	17

Merge Sort

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20

Merge:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	6	9	13	15	19	20	3	5	6	10	12	15	17

Merge Sort

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Output (sorted): **A**

1	3	4	5	6	6	9	10	12	13	15	15	17	19	20
---	---	---	---	---	---	---	----	----	----	----	----	----	----	----

Merge Sort

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ rearranged in non-decreasing order of value.

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. **MERGE-SORT** (A, p, q)
6. // recursively sort the right half
7. **MERGE-SORT** ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. **MERGE** (A, p, q, r)

Analyzing Divide-and-Conquer Algorithms

Let $T(n)$ be the running time of the algorithm on a problem of size n .

- If the problem size is small enough, say $n \leq c$ for some constant c , the straightforward solution takes $\Theta(1)$ time.
- Suppose our division of the problem yields a subproblems, each of which is $1/b$ the size of the original.
- Let $D(n)$ = time needed to divide the problem into subproblems.
- Let $C(n)$ = time needed to combine the solutions to the subproblems into the solution to the original problem.

$$\text{Then } T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT\left(\frac{n}{b}\right) + D(n) + C(n) & \text{otherwise.} \end{cases}$$

Analysis of Merge Sort

Let $T(n)$ be the worst-case running time of MERGE-SORT on n numbers.

We reason as follows to set up the recurrence for $T(n)$.

- When $n = 1$, MERGE-SORT takes $\Theta(1)$ time.
- When $n > 1$, we break down the running time as follows.
 - **Divide:** This step simply computes the middle of the subarray, which takes constant time. Hence, $D(n) = \Theta(1)$.
 - **Conquer:** We recursively solve 2 subproblems of size $n/2$ each, which adds $2T(n/2)$ to the running time.
 - **Combine:** The MERGE procedure takes $\Theta(n)$ time on an n -element subarray. Hence, $C(n) = \Theta(n)$.

$$\text{Then } T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Analysis of Merge Sort (Upper Bound)

Let us assume for simplicity that $n = 2^k$ for some integer $k \geq 0$, and for constants c_1 and c_2 :

$$T(n) \leq \begin{cases} c_1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + c_2n & \text{if } n > 1; \end{cases}$$

where, c_1 is an upper bound on the time needed to solve a problem of size 1, and c_2 is an upper bound on the time per array element of the divide and combine steps.

Let's see how the recursion unfolds.

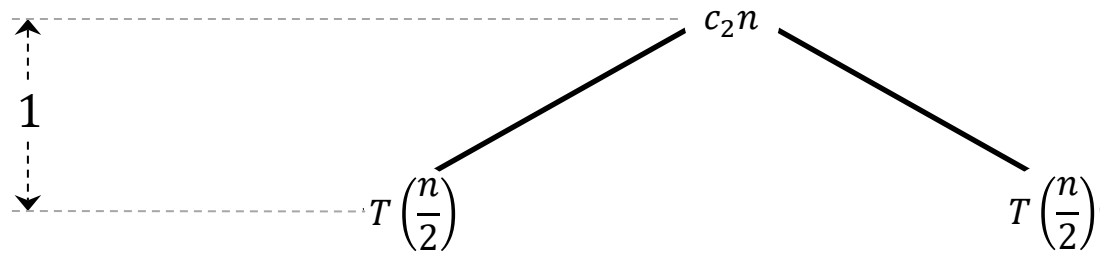
Analysis of Merge Sort (Upper Bound)

Running time on an input of size $n = 2^k$ for some integer $k \geq 0$:

$$T(n)$$

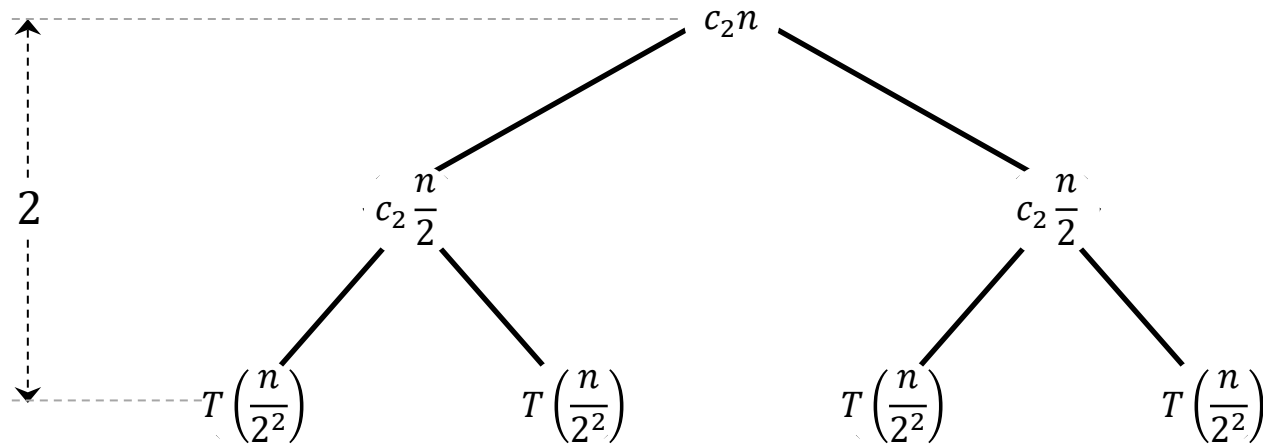
Analysis of Merge Sort (Upper Bound)

Unfolding the recurrence up to level 1:



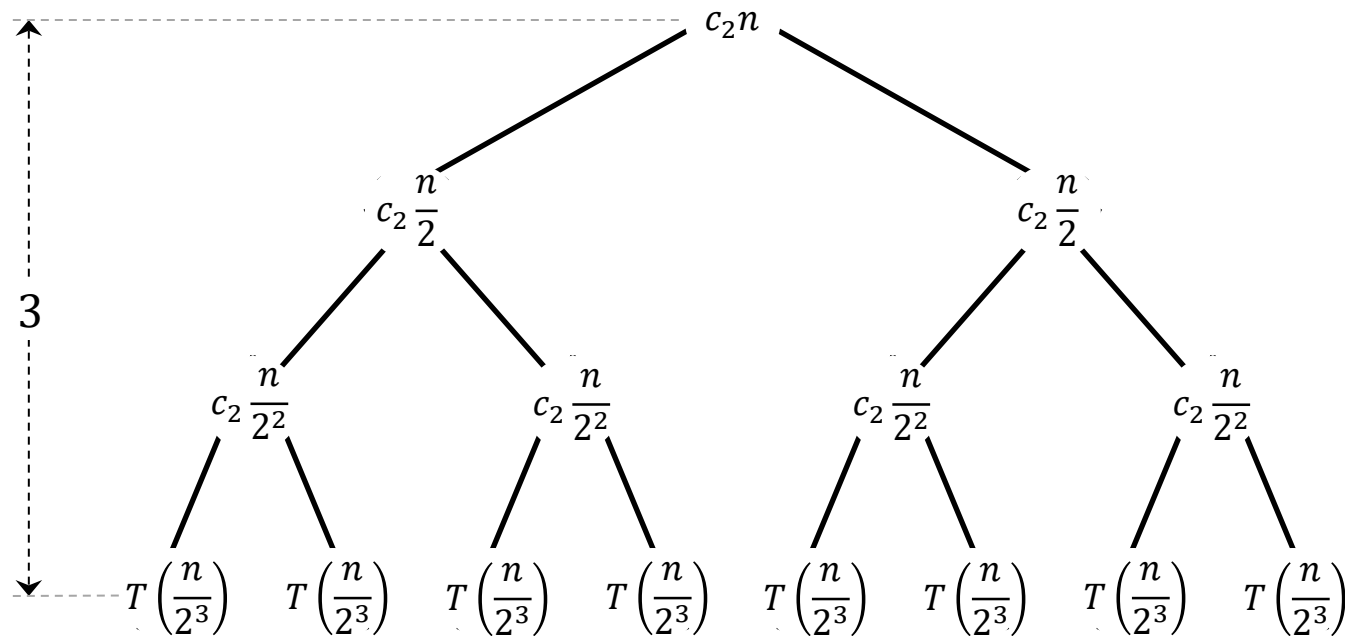
Analysis of Merge Sort (Upper Bound)

Unfolding the recurrence up to level 2:



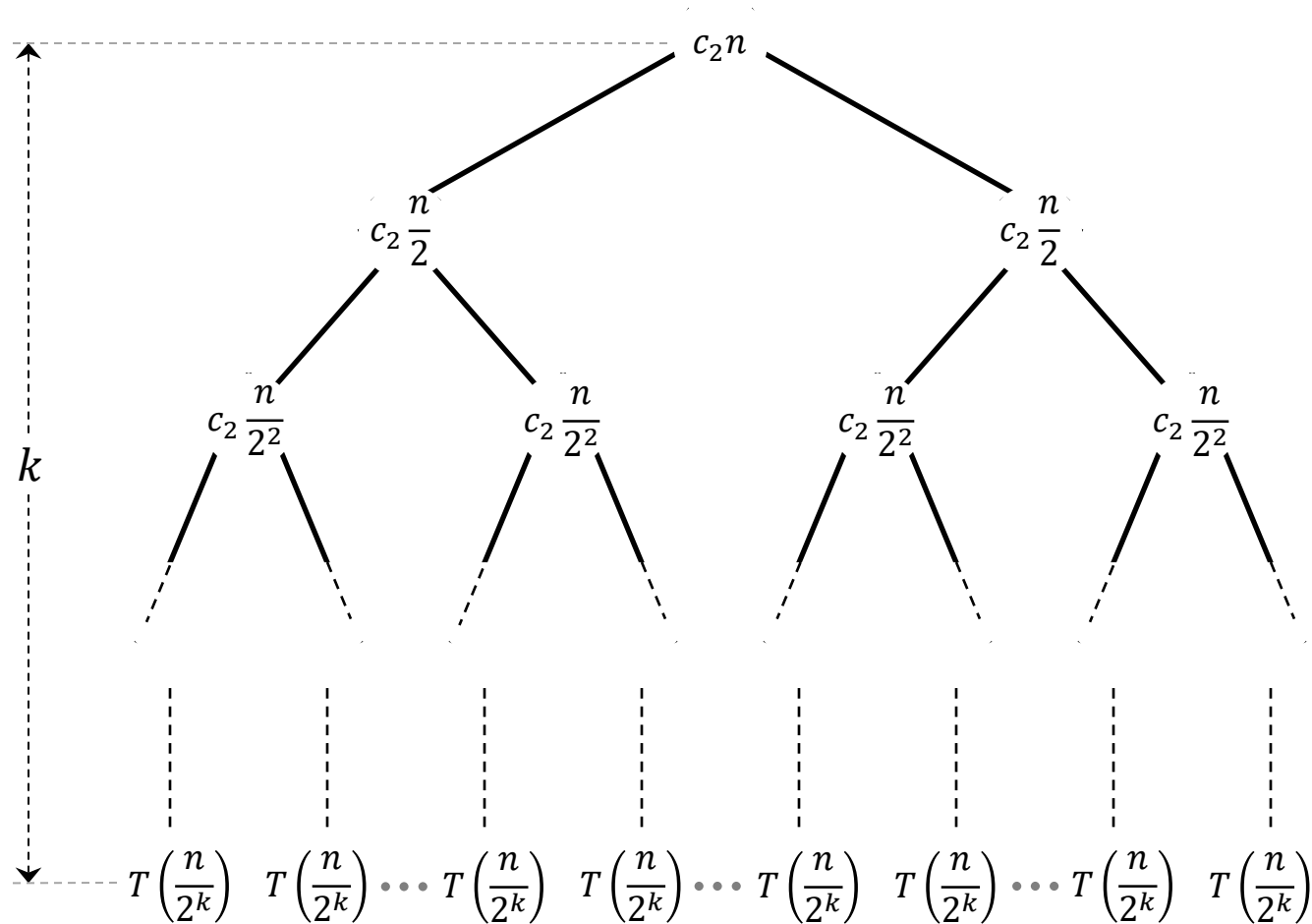
Analysis of Merge Sort (Upper Bound)

Unfolding the recurrence up to level 3:



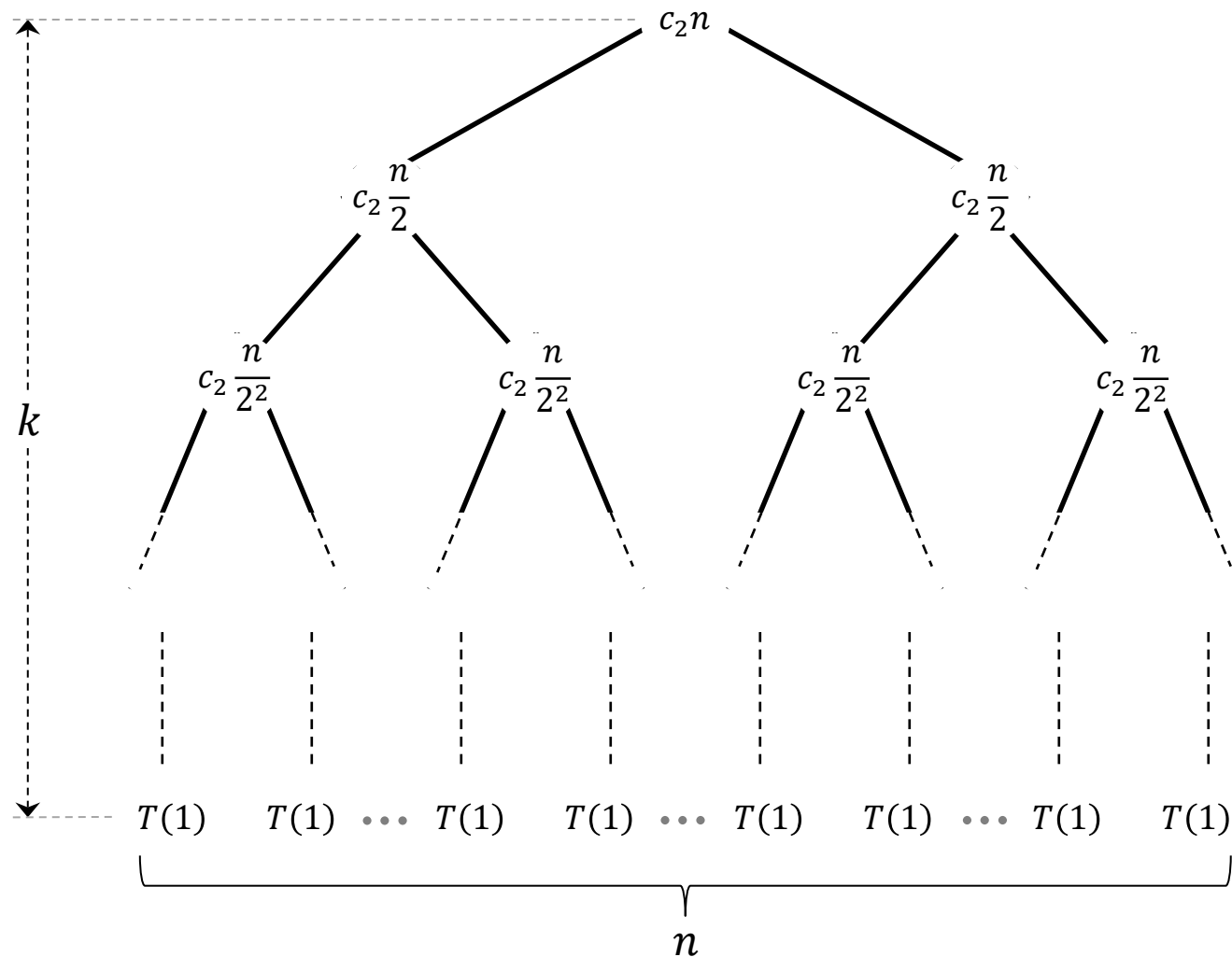
Analysis of Merge Sort (Upper Bound)

Unfolding the recurrence up to level k :



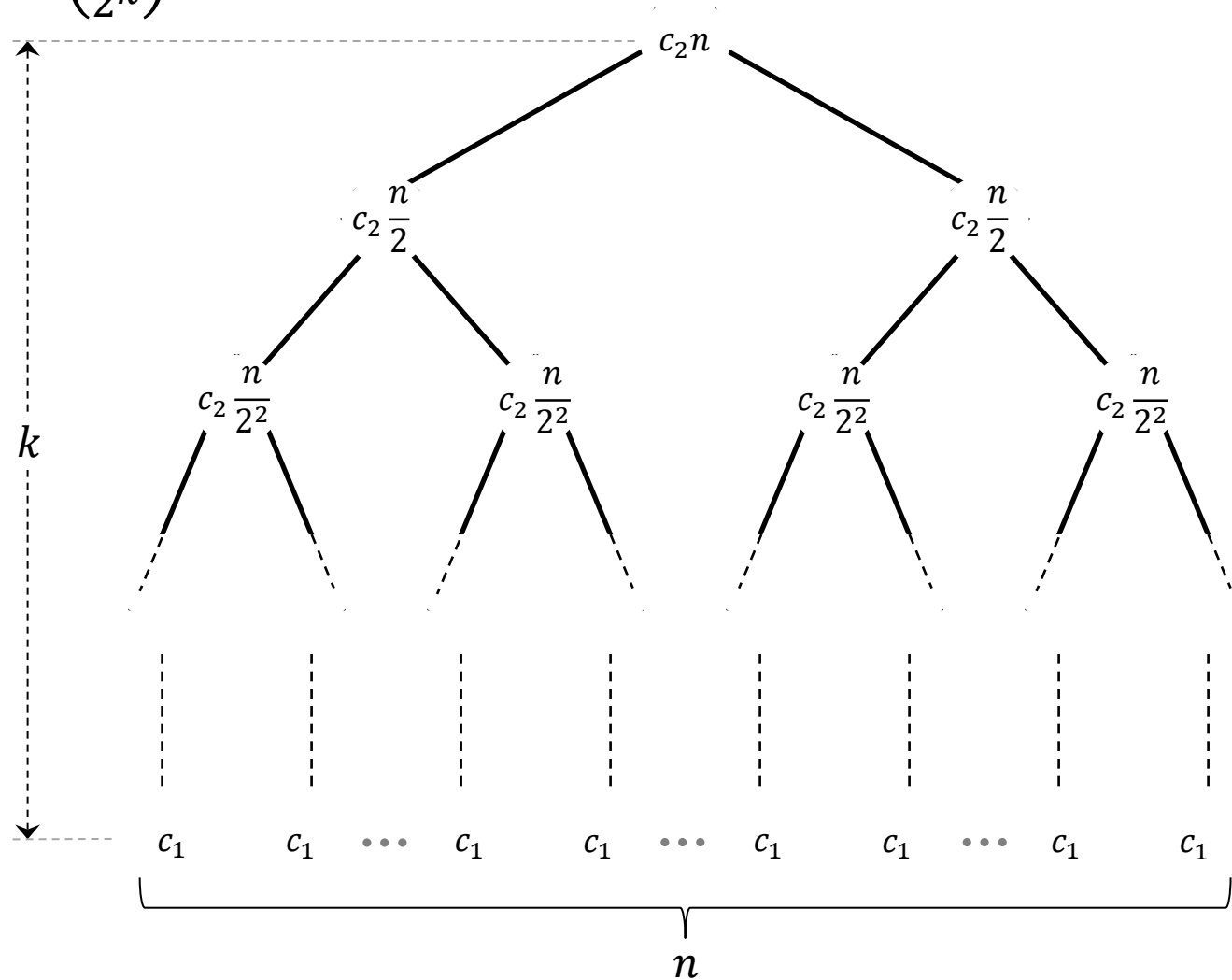
Analysis of Merge Sort (Upper Bound)

But $n = 2^k \Rightarrow \frac{n}{2^k} = 1$, and there will be n nodes (leaves) at level k :



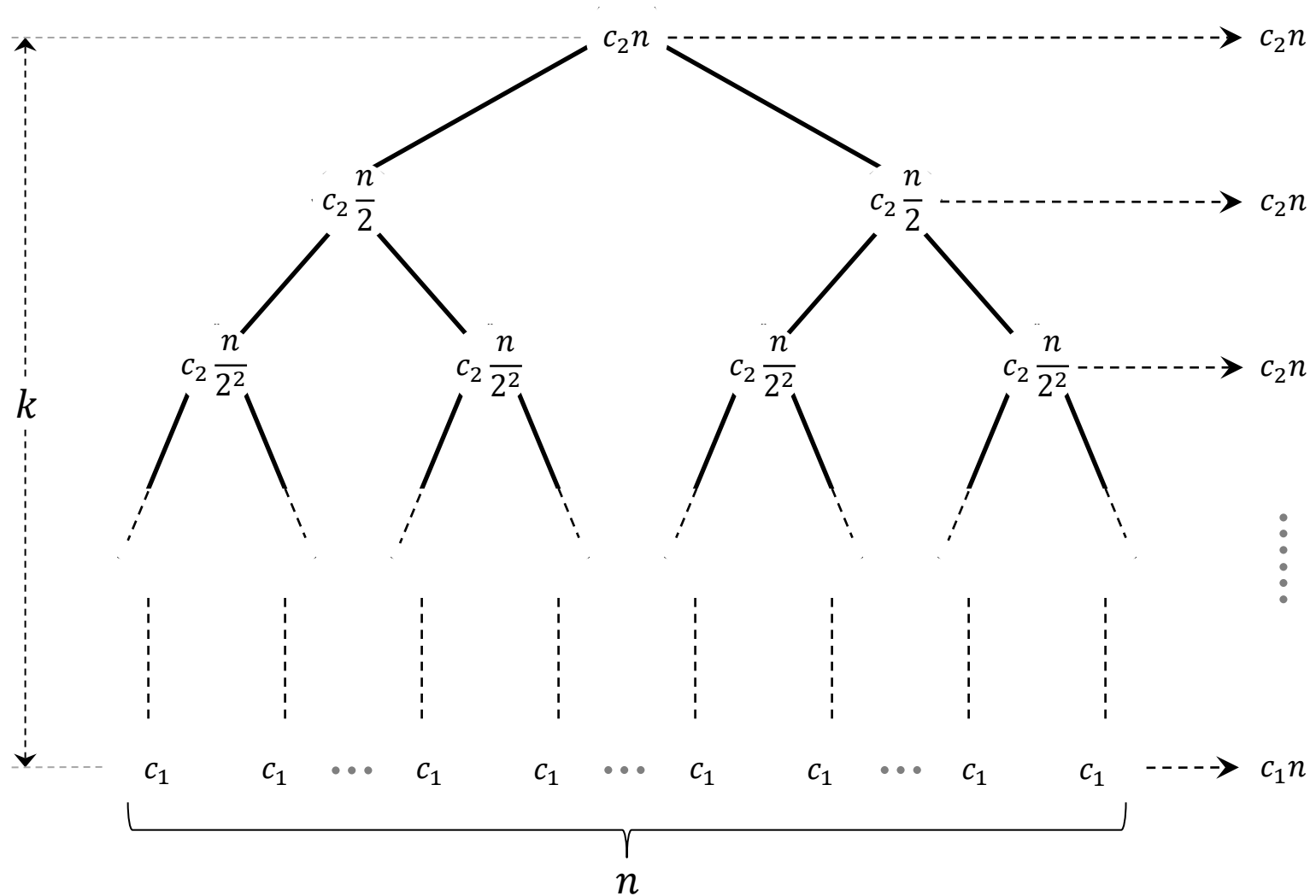
Analysis of Merge Sort (Upper Bound)

Then $T\left(\frac{n}{2^k}\right) = T(1) = c_1$:



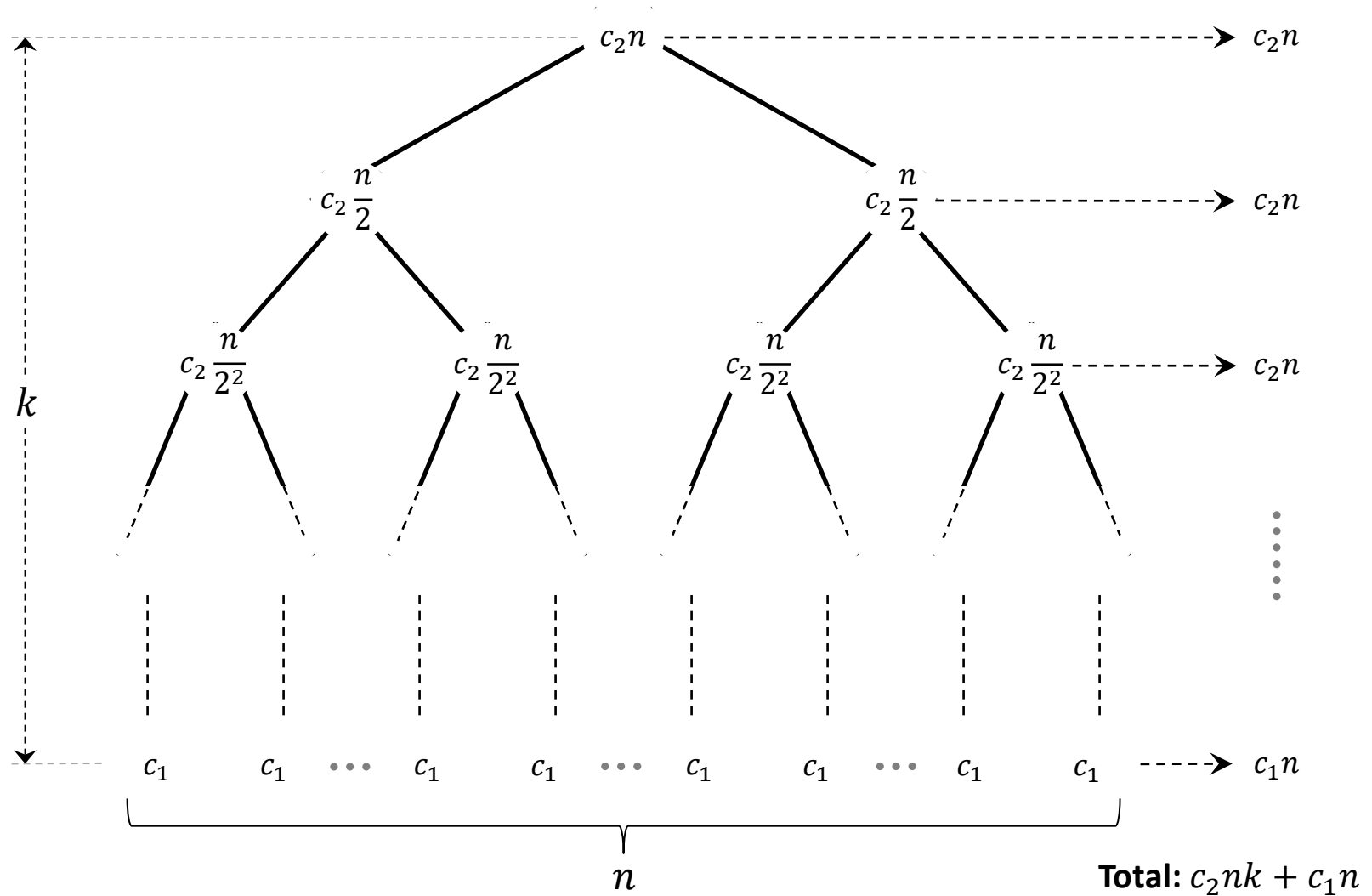
Analysis of Merge Sort (Upper Bound)

Total work at each level:



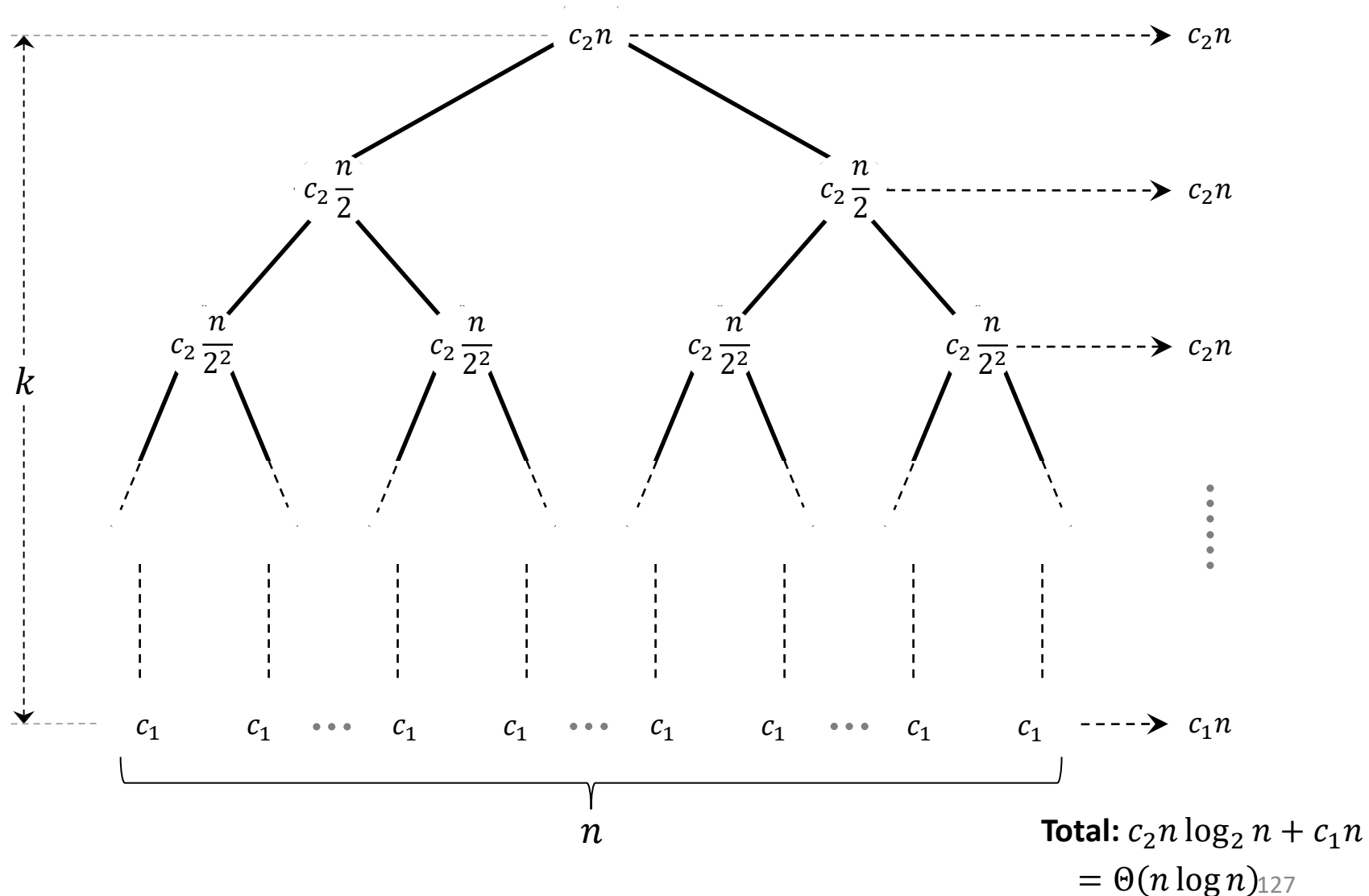
Analysis of Merge Sort (Upper Bound)

Total work across all levels:



Analysis of Merge Sort (Upper Bound)

But $n = 2^k \Rightarrow k = \log_2 n$:



Analysis of Merge Sort (Upper Bound)

Hence, we have:

$$T(n) \leq \Theta(n \log n)$$

Implying:

$$T(n) = O(n \log n)$$

Analysis of Merge Sort (Lower Bound)

Assuming $n = 2^k$ for some integer $k \geq 0$, for some constants c'_1 and c'_2 , we have:

$$T(n) \geq \begin{cases} c'_1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + c'_2 n & \text{if } n > 1; \end{cases}$$

where, c'_1 is a lower bound on the time needed to solve a problem of size 1, and c'_2 is a lower bound on the time per array element of the divide and combine steps.

Using the approach we used for proving the upper bound, we have:

$$T(n) \geq \Theta(n \log n)$$

Implying:

$$T(n) = \Omega(n \log n)$$

Analysis of Merge Sort (Tight Bound)

We have proved, upper bound: $T(n) = O(n \log n)$

and lower bound: $T(n) = \Omega(n \log n)$

Combining we get the tight bound:

$$T(n) = \Theta(n \log n)$$

Proof of Correctness of Merge Sort

Loop Invariants

We use *loop invariants* to prove correctness of iterative algorithms.

A loop invariant is associated with a given loop of an algorithm.

It is a formal statement about the relationship among variables of the algorithm such that

- [**Initialization**] It is true prior to the first iteration of the loop
- [**Maintenance**] If it is true before an iteration of the loop, it remains true before the next iteration
- [**Termination**] When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - q$
3. Let $L[1:n_1 + 1]$ and $R[1:n_2 + 1]$ be new arrays
4. **for** $i = 1$ **to** n_1
5. $L[i] = A[p + i - 1]$
6. **for** $j = 1$ **to** n_2
7. $R[j] = A[q + j]$
8. $L[n_1 + 1] = \infty$
9. $R[n_2 + 1] = \infty$
10. $i = 1$
11. $j = 1$
12. **for** $k = p$ **to** r
13. **if** $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i = i + 1$
16. **else** $A[k] = R[j]$
17. $j = j + 1$

Loop Invariant

At the start of each iteration of the **for** loop of lines 12–17 the following invariant holds:

The subarray $A[p : k - 1]$ contains the $k - p$ smallest elements of $L[1:n_1 + 1]$ and $R[1:n_2 + 1]$, in sorted order.

Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

Correctness of Merge Sort

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. **MERGE-SORT** (A, p, q)
6. // recursively sort the right half
7. **MERGE-SORT** ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. **MERGE** (A, p, q, r)

The proof has two parts.

- First, we will show that the algorithm terminates.
- Then we will show that the algorithm produces correct results (assuming the algorithm terminates).

Termination Guarantee

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

Size of the input subarray, $n = r - p + 1$

Size of the left half, $n_1 = q - p + 1$

Size of the right half, $n_2 = r - (q + 1) + 1 = r - q$

We will show the following: $n_1 < n$ and $n_2 < n$

Meaning: Sizes of subproblems decrease by at least 1 in each recursive call, and so there cannot be more than $n - 1$ levels of recursion. So, MERGE-SORT will terminate in finite time.

Termination Guarantee

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

A problem will be recursively subdivided (i.e., lines 5 and 7 will be executed) provided the following holds in line 1: $p < r$

But $p < r$ implies:

$$\begin{aligned} p + r < 2r &\Rightarrow \frac{p+r}{2} < r \Rightarrow \left\lfloor \frac{p+r}{2} \right\rfloor < r \\ &\Rightarrow q < r \Rightarrow q - p + 1 < r - p + 1 \Rightarrow n_1 < n \end{aligned}$$

Termination Guarantee

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

A problem will be recursively subdivided (i.e., lines 5 and 7 will be executed) provided the following holds in line 1: $p < r$

$p < r$ also implies:

$$2p < p + r \Rightarrow p < \frac{p+r}{2} \Rightarrow p \leq \left\lfloor \frac{p+r}{2} \right\rfloor \Rightarrow p \leq q$$

$$\Rightarrow -q \leq -p \Rightarrow r - q \leq r - p \Rightarrow r - q < r - p + 1 \Rightarrow n_2 < n$$

Inductive Proof of Correctness

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

Let $n = r - p + 1$.

Base Case: The algorithm is trivially correct when $r \geq p$, i.e., $n \leq 1$.

Inductive Hypothesis: Suppose the algorithm works correctly for all integral values of n not larger than k , where $k \geq 1$ is an integer.

Inductive Step: We will prove that the algorithm works correctly for $n = k + 1$.

Inductive Proof of Correctness

MERGE-SORT (A, p, r)

1. **if** $p < r$ **then**
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

When $n = k + 1$, lines 2–9 of the algorithm will be executed because $k \geq 1 \Rightarrow n > 1 \Rightarrow r - p + 1 > 1 \Rightarrow p < r$ holds in line 1.

The algorithm splits the input subarray $A[p:r]$ into two parts:

$A[p:q]$ and $A[q + 1:r]$, where $q = \left\lfloor \frac{p+r}{2} \right\rfloor$.

The recursive call in line 5 sorts the left part $A[p:q]$. Since $A[p:q]$ contains $n_1 = q - p + 1 < n \Rightarrow n_1 \leq k$ numbers, it is sorted correctly (using inductive hypothesis).

Inductive Proof of Correctness

MERGE-SORT (A, p, r)

1. if $p < r$ then
2. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
3. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
4. // recursively sort the left half
5. MERGE-SORT (A, p, q)
6. // recursively sort the right half
7. MERGE-SORT ($A, q + 1, r$)
8. // merge the two sorted halves and put the sorted sequence in $A[p..r]$
9. MERGE (A, p, q, r)

The recursive call in line 7 sorts the right part $A[q + 1:r]$. Since $A[q + 1:r]$ contains $n_2 = r - q < n \Rightarrow n_2 \leq k$ numbers, it is sorted correctly (using inductive hypothesis).

We know that the MERGE algorithm can merge two sorted arrays correctly. So, line 9 correctly merges the sorted left and right parts of the input subarray into a single sorted sequence in $A[p:q]$.

Therefore, the algorithm works correctly for $n = k + 1$, and consequently for all integral values of n .

*When the math
gets tough*

*Analysis of Algorithms
in
Rhymes and Pictures*

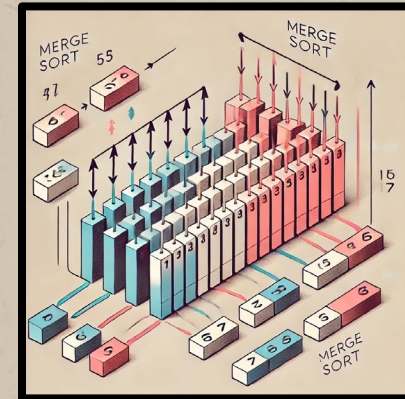
ChatGPT 4o mini

Merge Sort: A Rhyming Tale

When lists are long and sorting's slow,
Merge Sort's the way to make things flow.
It splits the list, it breaks apart,
Into halves, where each will start.

Divide, divide, until they're small,
Then merge them back—no fuss at all.
Each tiny piece will find its place,
Merging up with steady pace.

It conquers size, both big and wide,
Sorting's done from side to side.
And though the work might seem a chore,
It's faster than the ones before!



Here's an illustration of merge sort in action. The left side shows how the numbers or blocks are split into smaller parts, while the right side shows the merging process, with arrows showing how the pieces come back together in sorted order. It visualizes how merge sort works step by step!



Complexity Analysis in Rhyme

For **Best Case**, Worst, or **Average Day**,
Merge Sort's time is $O(n \log n)$ all the way.
It cuts and splits, no time to stall,
The $\log n$ makes it quick for all.

At each step, comparisons run,
 N times $\log n$ —efficiently done.
Though space it takes to hold the list,
In time, no better can exist.

With splitting, merging, sure and neat,
It sorts the data, fast and sweet.
So if your list is long and tall,
Merge Sort's the best for sorting all!

